



**DATA ENGINEERING &
ANALYTICS**

Kanchan Sharma

AE (319)

**B.SC. (PROG.) PHYSICAL
SCIENCE WITH
COMPUTER SCIENCE**

Data Engineering & Analytics (ELGE-1B)
GE-1 (Electronics)
PROGRAM LIST

LAB 1-2: Basic Data Analysis using PANDAS python library

Write programs to perform following functions on the given Student datasets using Pandas library:

Student Name	Physics	Chemistry	Maths
Aman	78	80	88
Rahul	67	77	95
Shreya	85	85	100
Vinita	96	90	90

StudentDataset1

Roll #	Student Results					
	Student	Department	General Sc.	Math's	Computer	Remarks
1	Agarwal V	Computer Sc	78	80	66	Pass
2	Agnew P	Computer Sc	45	78	66	Pass
3	Ahmad H	Statistics	78	30	50	Fail
4	Alambritis G	Physics	65	68	70	Pass
5	Allen J	Economics	87	50	60	Pass
6	Anthony A	Math's	54	30	30	Fail
7	Antoniou V	Computer Sc	76	70	70	?
8	Archy J	Economics	76	40	40	Fail
9	Armstrong B	Math's	86	30	40	?
10	Arvanitakis I	Physics	68	70	80	Pass

StudentDataset2

1. Creating Data frame from the given dataset by using DataFrame() function in pandas
2. Reading data (certain number of rows and columns) from a Data frame using **Locate Row**, loc[] and iloc[] function
3. Adding name to each row using '**index**' and to access data of a row using row index name
4. To access data of a student by specifying 'Student Name'
5. Reading student data from .xls/.xlsx file, .csv file and .json file
6. Determining the shape (rows, columns) of the dataset using shape function
7. Determining the size (no. of elements) in the given dataset using size function
8. Displaying first few rows/ last few rows of a dataframe using head/tail function
9. Displaying information about the type of data in a dataframe using info function
10. Listing names of all the columns in a dataset
11. Inserting a new column in an existing dataframe using/without using insert function
12. Inserting a new row in an existing dataframe using loc function
13. Concatenation of two dataframes using concat function
14. To evaluate percentage and display it in a separate column
15. Evaluation of the following statistical parameters using describe function: Count, Mean, Min, Max, Standard Deviation, 25th Percentile, 50th Percentile, 75th Percentile

LAB 3 - 4: Data Representation using Vectors and Matrices using NUMPY Library

1. Write a program to perform the following functions

$$A = \begin{bmatrix} -5 & 1 & -3 \\ 6 & 0 & 2 \\ 2 & 6 & 1 \end{bmatrix} \quad B = \begin{bmatrix} 2 & 4 & 5 \\ -8 & 10 & 3 \\ -2 & -3 & -9 \end{bmatrix}$$

- Create matrices A and B as shown in the attachment using array() function in numpy
- Write program to find add, subtract and multiply A and B
- Write a program to find the determinant of A and B

2. Determine the Rank and Nullity of matrices 1 to 5

$$1. \quad A = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 4 & 2 & 3 & 0 \\ 1 & 0 & 0 & 0 \\ 4 & 0 & 3 & 0 \end{bmatrix}$$

$$4. \quad A = \begin{bmatrix} 1 & 1 & 2 \\ 3 & 4 & -1 \\ -1 & -2 & 5 \end{bmatrix}$$

$$2. \quad Z = \begin{bmatrix} 5 & 4 & 7 \\ 5 & -6 & 5 \\ 4 & 2 & -3 \end{bmatrix}$$

$$5. \quad A = \begin{bmatrix} 1 & 3 & 4 \\ 3 & 9 & 12 \\ 1 & 3 & 4 \end{bmatrix}$$

$$3. \quad A = \begin{bmatrix} 1 & 0 & 1 \\ 2 & 1 & 0 \\ 0 & -1 & 2 \end{bmatrix}$$

3. Consider the following two vectors

$u = (0.5, 0.4, 0.4, 0.5, 0.1, 0.4, 0.1)$ and $v = (-1, -2, 1, -2, 3, 1, -5)$

- Check if u or v is a unit vector.
- Calculate the dot product, $\langle u, v \rangle$
- Are u and v orthogonal?

4. Consider the following three vectors

$v = (1, 2, 5, 2, -3, 1, 2, 6, 2)$

$u = (-4, 3, -2, 2, 1, -3, 4, 1, -2)$

$w = (3, 3, 3, -1, 6, -1, 2, -5, -7)$

- Evaluate dot product $\langle v, w \rangle$
- Are any pair of vectors orthogonal, and if so which ones?

5. Consider the following three matrices A, B and C

$$A = \begin{bmatrix} 2 & -2 \\ -3 & 1 \\ 5 & -3 \end{bmatrix} \quad B = \begin{bmatrix} 4 & 4 & 4 \\ -2 & 3 & -7 \\ 2 & 5 & -7 \end{bmatrix} \quad C = \begin{bmatrix} 4 & -1 & 2 \\ -8 & 2 & -4 \\ 2 & 1 & -4 \end{bmatrix}$$

Evaluate the following

- $A^T B$
- $C + B$
- Which matrices are full rank?
- B^{-1}

**Full Rank matrices are those for which Rank = No. of columns = No. of Rows*

LAB 5: Data Representation using Vectors and Matrices using NUMPY Library

1. Create the dataframe shown below and perform the following functions using PANDAS library

EMP ID	EMP NAME	SALARY	START DATE
1	Satish	50000	01-11-2017
2	Reeya	75000	12-05-2016
3	Jay	100000	22-09-2015
4	Rahul		11-10-2016
5	Roy	45000	08-01-2017
6	Jay	100000	22-09-2015
7	Vishal		05-01-2016
8	Serah	55000	06-02-2018

- Display the column names and the number of records.
- Display the first 4 records of the dataset.

- iii. For each numeric attribute, evaluate various statistical parameters using describe() function
 - iv. Check for the presence of missing values in the dataset and replace them with some valid numeric value
 - v. Find and remove duplicate records (if any) in the dataset.
2. Download **Pima Indians Diabetes Dataset** using the link given below:

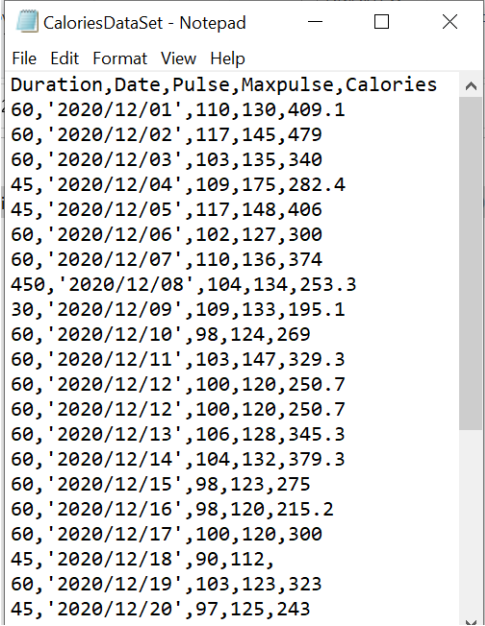
<https://www.kaggle.com/datasets/kumargh/pimaindiansdiabetescsv>

Perform the following operations:

- i. Display the column names and the number of records.
- ii. Display the first 10 records of the dataset.
- iii. For each numeric attribute, evaluate various statistical parameters using describe() function
- iv. Check for the presence of missing values in the dataset and replace them with some valid numeric value
- v. Find and remove duplicate records (if any) in the dataset.
- vi. Show scatter plot depicting relationship between two numeric columns of your choice.

LAB 6: Data Munging , Data Aggregation and Grouping Operations using PANDAS, MATPLOTLIB and SEABORN library)

1. Perform the following operations on the given dataset **CaloriesDataSet.csv**



```

CaloriesDataSet - Notepad
File Edit Format View Help
Duration,Date,Pulse,Maxpulse,Calories
60,'2020/12/01',110,130,409.1
60,'2020/12/02',117,145,479
60,'2020/12/03',103,135,340
45,'2020/12/04',109,175,282.4
45,'2020/12/05',117,148,406
60,'2020/12/06',102,127,300
60,'2020/12/07',110,136,374
450,'2020/12/08',104,134,253.3
30,'2020/12/09',109,133,195.1
60,'2020/12/10',98,124,269
60,'2020/12/11',103,147,329.3
60,'2020/12/12',100,120,250.7
60,'2020/12/12',100,120,250.7
60,'2020/12/13',106,128,345.3
60,'2020/12/14',104,132,379.3
60,'2020/12/15',98,123,275
60,'2020/12/16',98,120,215.2
60,'2020/12/17',100,120,300
45,'2020/12/18',90,112,
60,'2020/12/19',103,123,323
45,'2020/12/20',97,125,243
  
```

- Loading the Dataset
- Implement descriptive and summary statistics (to calculate Count, Mean, Max and Min, Percentile, Variance and Standard Deviation)
- Perform the following **DATA MUNGING and DATA CLEANING** operations (using PANDAS):
 - Check for the presence of missing values in the dataset and replace them with some valid numeric value
 - Find and remove duplicate records (if any) in the dataset.
 - Determine the correlation matrix for different columns (attributes) in a given dataset
- Perform the following **DATA VISUALISATION** operations (using Matplotlib and seaborn library)
 - Plot histogram, bar plot, distplot for various features attributes of the dataset
 - Plot Heatmap for the correlation between different attributes in the dataset
- Perform the following **DATA AGGREGATION and Grouping** operations:
 - using agg(), aggregate function to calculate sum, min and max of each column
 - group the dataset as per 'Duration' column

- display the number (count) of values for each 'Duration'
 - display the sum of all the values for each 'Duration'
 - perform various Data Aggregation functions
 - perform various Data Aggregation functions for a particular column (attribute)
2. Download **Pima Indians Diabetes Dataset** using the link given below:
<https://www.kaggle.com/datasets/kumargh/pimaindiandabetescsv>
 Perform similar Data Munging and Data Aggregation operations on the **Pima Indians Diabetes Dataset** as performed on CaloriesDataSet.csv
 3. Download **Boston House Prices Dataset** using the link given below:
<https://www.kaggle.com/datasets/vikrishnan/boston-house-prices>
 Perform similar Data Munging and Data Aggregation operations on the **Boston House Prices Dataset** as performed on CaloriesDataSet.csv in Q.1

LAB 7: TIME-SERIES DATA ANALYSIS: Analysis of Time-dependent data to predict Future Trends from Past Values STATS MODEL- PYTHON LIBRARY

1. Perform the following operations on the given Time-series dataset **AirPassengers.csv**

AirPassengers		
	A	B
1	Month	#Passengers
2	1949-01	112
3	1949-02	118
4	1949-03	132
5	1949-04	129
6	1949-05	121
7	1949-06	135
8	1949-07	148
9	1949-08	148
10	1949-09	136
11	1949-10	119
12	1949-11	104
13	1949-12	118
14	1950-01	115
15	1950-02	126
16	1950-03	141
17	1950-04	135
18	1950-05	125
19	1950-06	149
20	1950-07	170

- Reading Time-series data into Pandas dataframe
 - PLOTTING TIME-SERIES DATA using plot() function
 - ETS Decomposition-MULTIPLICATIVE MODEL
 - Extracting 'TREND' component
 - Extracting 'SEASONAL' component
 - Extracting 'RESIDUAL' component
 - ETS Decomposition-ADDITIVE MODEL
 - Extracting 'TREND' component
 - Extracting 'SEASONAL' component
 - Extracting 'RESIDUAL' component
2. Download **SuperStore-SalesDataSet** using the link given below:
<https://www.kaggle.com/datasets/rohitsahoo/sales-forecasting>
 Perform similar operations on the dataset as performed on **AirPassengers.csv** dataset

```
# Program to print single line text message

print("Welcome to Python Programming")

    Welcome to Python Programming

# Program to print a multi-line text message

print("Hello!!\nGood Afternoon !\nWelcome to AI Lab")

    Hello!!
    Good Afternoon !
    Welcome to AI Lab

# Datatypes (various types of data recognised by python programming language)
# declaration of variables (Variable- named memory location which stores data)

# INTEGER type variable

a=5
b=5
print(a+b)
type(a)

# CHARACTER type variable

ch='5'
type(ch)
print(ch)
print(int(ch))

# FLOAT type variable

d= 5.2
type(d)
print(int(d))

10
5
5
5

# CHARACTER type variable

ch='5'
#type(ch)
#print(ch)
print(int(ch))

5
```

```
# Declaring and displaying a string
```

```
name="XYZ"  
print(name)
```

```
# To add a integer with a decimal number (float type variable)
```

```
a=7  
b=4.5  
c= a+b  
print("Addition of a & b is",c)          #displaying result as a decimal number  
print("Addition of a & b is",int(c))      #displaying result as an integer
```

```
    Addition of a & b is 11.5  
    Addition of a & b is 11
```

```
# Boolean type variable
```

```
s =True  
print(s)  
type(s)
```

```
    True  
    bool
```

```
a=10  
print(a==10)
```

```
    True
```

```
# INPUT statement to accept data as input from the user
```

```
name= input("Please enter your name")  
print(name)  
print("Hello", name,"\nGood Day\nWelcome to Python Programming")
```

```
    Please enter your nameMonika  
    Monika  
    Hello Monika  
    Good Day  
    Welcome to Python Programming
```

```
# Program to accept two real(decimal) numbers as input  
# add them and display the output
```

```
a = float(input("Please enter the first number"))  
b = float(input("Please enter the second number"))  
c=a+b  
print("addition is",c)
```

```
Please enter the first number4.5
Please enter the second number2
addition is 6.5
```

```
# Program to find the greatest of two numbers
```

```
a = float(input("Please enter the first number"))
b = float(input("Please enter the second number"))
```

```
if a>b:
    print(a,"is greater than",b)
    print(a)
elif b>a:
    print(b,"is greater than",a)
    print(b)
else:
    print(a,"is equal to",b)
```

```
Please enter the first number4
Please enter the second number4
4.0 is equal to 4.0
```

```
# Program to swap two numbers (using third variable)
```

```
a=3
b=2
print("a=",a)
print("b=",b)
c=a
a=b
b=c
print("a=",a)
print("b=",b)
```

```
a= 3
b= 2
a= 2
b= 3
```

```
# Program to swap two numbers (without using third variable)
```

```
a=3
b=2
print("a=",a)
print("b=",b)
(a,b)=(b,a)
print("a=",a)
print("b=",b)
```


[Colab paid products](#) - [Cancel contracts here](#)



#1. Creating a DataFrame from the given dataset using DataFrame()

```
import pandas as pd
```

```
StudentData = {'Roll':[1,2,3,4,5,6,7,8,9,10],
               'Student Name':['Agarwal V','Agnew P','Ahmad H','Alambritis G','Allen J','Anthony A','Antoniou V','Archy J','Armstrong B','Arv
               'Department':['Computer Sc','Computer Sc','Statistics','Physics','Economics','Math's','Computer Sc','Economics','Math's','Phys
               'General Sc':[78,45,78,65,87,54,76,76,86,68],
               'Maths': [80,78,30,68,50,30,70,40,30,70],
               'Remarks':['Pass','Pass','Fail','Pass','Pass','Fail','?','Fail','?','Pass']}
STUDENT_DATA = pd.DataFrame(StudentData)
print(STUDENT_DATA) #printing Data Frame (Data Table)
```

	Roll	Student Name	Department	General Sc	Maths	Remarks
0	1	Agarwal V	Computer Sc	78	80	Pass
1	2	Agnew P	Computer Sc	45	78	Pass
2	3	Ahmad H	Statistics	78	30	Fail
3	4	Alambritis G	Physics	65	68	Pass
4	5	Allen J	Economics	87	50	Pass
5	6	Anthony A	Math's	54	30	Fail
6	7	Antoniou V	Computer Sc	76	70	?
7	8	Archy J	Economics	76	40	Fail
8	9	Armstrong B	Math's	86	30	?
9	10	Arvanitakis I	Physics	68	70	Pass

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

#2. To determine details of student using loc function.

```
import pandas as pd
```

```
StudentData = {'Roll':[1,2,3,4,5,6,7,8,9,10],
               'Student Name':['Agarwal V','Agnew P','Ahmad H','Alambritis G','Allen J','Anthony A','Antoniou V','Archy J','Armstrong B','Arv
               'Department':['Computer Sc','Computer Sc','Statistics','Physics','Economics','Math's','Computer Sc','Economics','Math's','Phys
               'General Sc':[78,45,78,65,87,54,76,76,86,68],
               'Maths': [80,78,30,68,50,30,70,40,30,70],
               'Computer':[66,66,50,70,60,30,70,40,40,80],
               'Remarks':['Pass','Pass','Fail','Pass','Pass','Fail','?','Fail','?','Pass']}
STUDENT_DATA = pd.DataFrame(StudentData)
print(STUDENT_DATA.loc[[0]],'\n','\n',
STUDENT_DATA.iloc[0])
```

	Roll	Student Name	Department	General Sc	Maths	Computer	Remarks
0	1	Agarwal V	Computer Sc	78	80	66	Pass

```
Roll      1
Student Name  Agarwal V
Department    Computer Sc
General Sc    78
Maths         80
Computer      66
Remarks      Pass
Name: 0, dtype: object
```

#3, To give a name to each row

```
StudentData = {'Roll':[1,2,3,4,5,6,7,8,9,10],
               'Student Name':['Agarwal V','Agnew P','Ahmad H','Alambritis G','Allen J','Anthony A','Antoniou V','Archy J','Armstrong B','Arv
               'Department':['Computer Sc','Computer Sc','Statistics','Physics','Economics','Math's','Computer Sc','Economics','Math's','Phys
               'General Sc':[78,45,78,65,87,54,76,76,86,68],
               'Maths': [80,78,30,68,50,30,70,40,30,70],
               'Computer':[66,66,50,70,60,30,70,40,40,80],
               'Remarks':['Pass','Pass','Fail','Pass','Pass','Fail','?','Fail','?','Pass']}
STUDENT_DATA = pd.DataFrame(StudentData,index = ['student1','student2','student3','student4','student5','student6','student7','student
print(STUDENT_DATA)
```

	Roll	Student Name	Department	General Sc	Maths	Computer	\
student1	1	Agarwal V	Computer Sc	78	80	66	
student2	2	Agnew P	Computer Sc	45	78	66	
student3	3	Ahmad H	Statistics	78	30	50	
student4	4	Alambritis G	Physics	65	68	70	

student5	5	Allen J	Economics	87	50	60
student6	6	Anthony A	Math's	54	30	30
student7	7	Antoniou V	Computer Sc	76	70	70
student8	8	Archy J	Economics	76	40	40
student9	9	Armstrong B	Math's	86	30	40
student10	10	Arvanitakis I	Physics	68	70	80

	Remarks
student1	Pass
student2	Pass
student3	Fail
student4	Pass
student5	Pass
student6	Fail
student7	?
student8	Fail
student9	?
student10	Pass

#4. To access data of a row using row index name

```
StudentData = {'Roll':[1,2,3,4,5,6,7,8,9,10],
               'Student Name':['Agarwal V','Agnew P','Ahmad H','Alambritis G','Allen J','Anthony A','Antoniou V','Archy J','Armstrong B','Arv
               'Department':['Computer Sc','Computer Sc','Statistics','Physics','Economics','Math's','Computer Sc','Economics','Math's','Phys
               'General Sc':[78,45,78,65,87,54,76,76,86,68],
               'Maths': [80,78,30,68,50,30,70,40,30,70],
               'Computer':[66,66,50,70,60,30,70,40,40,80],
               'Remarks':['Pass','Pass','Fail','Pass','Pass','Fail','?','Fail','?','Pass']}

STUDENT_DATA = pd.DataFrame(StudentData,index = ['student1', 'student2', 'student3', 'student4', 'student5', 'student6', 'student7', 'student
print(STUDENT_DATA)
print(STUDENT_DATA.loc[['student1']])
```

	Roll	Student Name	Department	General Sc	Maths	Computer	\
student1	1	Agarwal V	Computer Sc	78	80	66	
student2	2	Agnew P	Computer Sc	45	78	66	
student3	3	Ahmad H	Statistics	78	30	50	
student4	4	Alambritis G	Physics	65	68	70	
student5	5	Allen J	Economics	87	50	60	
student6	6	Anthony A	Math's	54	30	30	
student7	7	Antoniou V	Computer Sc	76	70	70	
student8	8	Archy J	Economics	76	40	40	
student9	9	Armstrong B	Math's	86	30	40	
student10	10	Arvanitakis I	Physics	68	70	80	

	Remarks
student1	Pass
student2	Pass
student3	Fail
student4	Pass
student5	Pass
student6	Fail
student7	?
student8	Fail
student9	?
student10	Pass

	Roll	Student Name	Department	General Sc	Maths	Computer	Remarks
student1	1	Agarwal V	Computer Sc	78	80	66	Pass

#5. To determine details of a student 'Agarwal' by specifying student name in loc ()

```
import pandas as pd
```

```
StudentData = {'Roll':[1,2,3,4,5,6,7,8,9,10],
               'Student Name':['Agarwal V','Agnew P','Ahmad H','Alambritis G','Allen J','Anthony A','Antoniou V','Archy J','Armstrong B','Arv
               'Department':['Computer Sc','Computer Sc','Statistics','Physics','Economics','Math's','Computer Sc','Economics','Math's','Phys
               'General Sc':[78,45,78,65,87,54,76,76,86,68],
               'Maths': [80,78,30,68,50,30,70,40,30,70],
               'Computer':[66,66,50,70,60,30,70,40,40,80],
               'Remarks':['Pass','Pass','Fail','Pass','Pass','Fail','?','Fail','?','Pass']}

STUDENT_DATA = pd.DataFrame(StudentData)
STUDENT_DATA = STUDENT_DATA.set_index(['Student Name'])
print(STUDENT_DATA.loc[['Agarwal V']])
```

	Roll	Department	General Sc	Maths	Computer	Remarks
Student Name						
Agarwal V	1	Computer Sc	78	80	66	Pass

#6. Reading data from a spreadsheet (.xlsx/.xls) file

```
import pandas as pd
```

```
STUDENT_DATA = pd.read_excel('/content/drive/MyDrive/StudentDataSet2_LAB2.xlsx')
```

```
print(STUDENT_DATA)
```

	Roll	Student	Department	General Sc.	Math's	Computer	Remarks
0	1	Agarwal V	Computer Sc	78	80	66	Pass
1	2	Agnew P	Computer Sc	45	78	66	Pass
2	3	Ahmad H	Statistics	78	30	50	Fail
3	4	Alambritis G	Physics	65	68	70	Pass
4	5	Allen J	Economics	87	50	60	Pass
5	6	Anthony A	Math's	54	30	30	Fail
6	7	Antoniou V	Computer Sc	76	70	70	?
7	8	Archy J	Economics	76	40	40	Fail
8	9	Armstrong B	Math's	86	30	40	?
9	10	Arvanitakis I	Physics	68	70	80	Pass

```
#7. Reading data from a JSON format file (.json)
```

```
import pandas as pd
```

```
STUDENT_DATA = pd.read_json('/content/drive/MyDrive/StudentDataSet2_LAB2.json')
```

```
print(STUDENT_DATA)
```

	Roll	Student	Department	General Sc.	Maths	Computer	Remarks
0	1	Agarwal V	Computer Sc	78	80	66	Pass
1	2	Agnew P	Computer Sc	45	78	66	Pass
2	3	Ahmad H	Statistics	78	30	50	Fail
3	4	Alambritis G	Physics	65	68	70	Pass
4	5	Allen J	Economics	87	50	60	Pass
5	6	Anthony A	Math's	54	30	30	Fail
6	7	Antoniou V	Computer Sc	76	70	70	?
7	8	Archy J	Economics	76	40	40	Fail
8	9	Armstrong B	Math's	86	30	40	?
9	10	Arvanitakis I	Physics	68	70	80	Pass

```
#8. Reading data from a file (.csv)
```

```
import pandas as pd
```

```
STUDENT_DATA = pd.read_csv('/content/drive/MyDrive/StudentDataSet2_LAB2.csv')
```

```
print(STUDENT_DATA)
```

	Roll	Student	Department	General Sc.	Maths	Computer	Remarks
0	1	Agarwal V	Computer Sc	78	80	66	Pass
1	2	Agnew P	Computer Sc	45	78	66	Pass
2	3	Ahmad H	Statistics	78	30	50	Fail
3	4	Alambritis G	Physics	65	68	70	Pass
4	5	Allen J	Economics	87	50	60	Pass
5	6	Anthony A	Math's	54	30	30	Fail
6	7	Antoniou V	Computer Sc	76	70	70	?
7	8	Archy J	Economics	76	40	40	Fail
8	9	Armstrong B	Math's	86	30	40	?
9	10	Arvanitakis I	Physics	68	70	80	Pass

```
#9. To determine the shape ( no.of ROWS x no. of COLUMNS) in a given data frame
```

```
print("(ROWS, COLUMNS) in the given data set is", STUDENT_DATA.shape)
```

```
(ROWS, COLUMNS) in the given data set is (10, 7)
```

```
#10. To determine the size ( no.of data elements) in a given data frame
```

```
print("No. of elements in the given data set are", STUDENT_DATA.size)
```

```
No. of elements in the given data set are 70
```

```
#11. Displaying first few rows of a dataframe
```

```
import pandas as pd
```

```
STUDENT_DATA = pd.read_csv('/content/drive/MyDrive/StudentDataSet2_LAB2.csv')
```

```
print(STUDENT_DATA.head(2))
```

	Roll	Student	Department	General Sc.	Maths	Computer	Remarks
0	1	Agarwal V	Computer Sc	78	80	66	Pass
1	2	Agnew P	Computer Sc	45	78	66	Pass

#12. Displaying last few rows of a dataframe

```
import pandas as pd
```

```
STUDENT_DATA = pd.read_csv('/content/drive/MyDrive/StudentDataSet2_LAB2.csv')
```

```
print(STUDENT_DATA.tail(2))
```

	Roll	Student	Department	General Sc.	Maths	Computer	Remarks
8	9	Armstrong B	Math's	86	30	40	?
9	10	Arvanitakis I	Physics	68	70	80	Pass

#13. Displaying information about the data

```
import pandas as pd
```

```
STUDENT_DATA = pd.read_csv('/content/drive/MyDrive/StudentDataSet2_LAB2.csv')
```

```
print(STUDENT_DATA.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Roll                  10 non-null    int64
1   Student               10 non-null    object
2   Department            10 non-null    object
3   General Sc.          10 non-null    int64
4   Maths                 10 non-null    int64
5   Computer              10 non-null    int64
6   Remarks              10 non-null    object
dtypes: int64(4), object(3)
memory usage: 688.0+ bytes
None
```

#14. To display column names

```
import pandas as pd
```

```
STUDENT_DATA = pd.read_csv('/content/drive/MyDrive/StudentDataSet2_LAB2.csv')
```

```
print(STUDENT_DATA.columns)
```

```
Index(['Roll', 'Student', 'Department', 'General Sc.', 'Maths', 'Computer',
      'Remarks'],
      dtype='object')
```

#15. To add a new column using insert function

```
import pandas as pd
```

```
STUDENT_DATA = pd.read_csv('/content/drive/MyDrive/StudentDataSet2_LAB2.csv')
```

```
STUDENT_DATA.insert(4, 'English', [96, 75, 88, 98, 89, 90, 56, 87, 68, 80], True)
```

```
print(STUDENT_DATA)
```

	Roll	Student	Department	General Sc.	English	Maths	Computer	\
0	1	Agarwal V	Computer Sc	78	96	80	66	
1	2	Agnew P	Computer Sc	45	75	78	66	
2	3	Ahmad H	Statistics	78	88	30	50	
3	4	Alambritis G	Physics	65	98	68	70	
4	5	Allen J	Economics	87	89	50	60	
5	6	Anthony A	Math's	54	90	30	30	
6	7	Antoniou V	Computer Sc	76	56	70	70	
7	8	Archy J	Economics	76	87	40	40	
8	9	Armstrong B	Math's	86	68	30	40	
9	10	Arvanitakis I	Physics	68	80	70	80	

	Remarks
0	Pass
1	Pass
2	Fail
3	Pass
4	Pass

```

5   Fail
6   ?
7   Fail
8   ?
9   Pass

```

#16. To add a new column without using insert function

```
import pandas as pd
```

```
STUDENT_DATA = pd.read_csv('/content/drive/MyDrive/StudentDataSet2_LAB2.csv')
```

```
English = [96, 75, 88, 98,89,90,56,87,68,80]
```

```
STUDENT_DATA['English']=English
```

```
print(STUDENT_DATA)
```

	Roll	Student	Department	General Sc.	Maths	Computer	Remarks	\
0	1	Agarwal V	Computer Sc	78	80	66	Pass	
1	2	Agnew P	Computer Sc	45	78	66	Pass	
2	3	Ahmad H	Statistics	78	30	50	Fail	
3	4	Alambritis G	Physics	65	68	70	Pass	
4	5	Allen J	Economics	87	50	60	Pass	
5	6	Anthony A	Math's	54	30	30	Fail	
6	7	Antoniou V	Computer Sc	76	70	70	?	
7	8	Archy J	Economics	76	40	40	Fail	
8	9	Armstrong B	Math's	86	30	40	?	
9	10	Arvanitakis I	Physics	68	70	80	Pass	

	English
0	96
1	75
2	88
3	98
4	89
5	90
6	56
7	87
8	68
9	80

#17. To add a new row using loc function

```
import pandas as pd
```

```
STUDENT_DATA = pd.read_csv('/content/drive/MyDrive/StudentDataSet2_LAB2.csv')
```

```
STUDENT_DATA.loc[10]=[11,'Vishal',"Computer Sc",88,97,100,"Pass"]
```

```
print(STUDENT_DATA)
```

	Roll	Student	Department	General Sc.	Maths	Computer	Remarks
0	1	Agarwal V	Computer Sc	78	80	66	Pass
1	2	Agnew P	Computer Sc	45	78	66	Pass
2	3	Ahmad H	Statistics	78	30	50	Fail
3	4	Alambritis G	Physics	65	68	70	Pass
4	5	Allen J	Economics	87	50	60	Pass
5	6	Anthony A	Math's	54	30	30	Fail
6	7	Antoniou V	Computer Sc	76	70	70	?
7	8	Archy J	Economics	76	40	40	Fail
8	9	Armstrong B	Math's	86	30	40	?
9	10	Arvanitakis I	Physics	68	70	80	Pass
10	11	Vishal	Computer Sc	88	97	100	Pass

#18. To concatenate two dataframes

```
import pandas as pd
```

```
StudentData1 = {'Roll':[1,2,3,4,5,6,7,8,9,10],
                'Student Name':['Agarwal V','Agnew P','Ahmad H','Alambritis G','Allen J','Anthony A','Antoniou V','Archy J','Armstrong B','Arv
                'Department':['Computer Sc','Computer Sc','Statistics','Physics','Economics','Math's','Computer Sc','Economics','Math's','Phys
                'General Sc':[78,45,78,65,87,54,76,76,86,68],
                'Maths': [80,78,30,68,50,30,70,40,30,70],
                'Computer':[66,66,50,70,60,30,70,40,40,80],
                'Remarks':['Pass','Pass','Fail','Pass','Pass','Fail','?','Fail','?','Pass']}
```

```
StudentData2 = {'Roll':[11,12,13,14,15],
                'Student Name':['Ashish k','Shreya','jitendar','Vinit','Anuj'],
                'Department':['Math's','Computer Sc','Economics','Math's','Physics'],
                'General Sc':[78,45,78,65,87,],
                'Maths': [80,78,30,68,50],
```

```
'Computer':[66,66,50,70,60,],
'Remarks':['Pass','Pass','Fail','Pass','Pass']}]}
```

```
STUDENT_DATA1=pd.DataFrame(StudentData1)
STUDENT_DATA2=pd.DataFrame(StudentData2)
```

```
STUDENT_DATA = pd.concat([STUDENT_DATA1,STUDENT_DATA2], ignore_index=True)
print(STUDENT_DATA)
```

	Roll	Student Name	Department	General Sc	Maths	Computer	Remarks
0	1	Agarwal V	Computer Sc	78	80	66	Pass
1	2	Agnew P	Computer Sc	45	78	66	Pass
2	3	Ahmad H	Statistics	78	30	50	Fail
3	4	Alambritis G	Physics	65	68	70	Pass
4	5	Allen J	Economics	87	50	60	Pass
5	6	Anthony A	Math's	54	30	30	Fail
6	7	Antoniou V	Computer Sc	76	70	70	?
7	8	Archy J	Economics	76	40	40	Fail
8	9	Armstrong B	Math's	86	30	40	?
9	10	Arvanitakis I	Physics	68	70	80	Pass
10	11	Ashish k	Math's	78	80	66	Pass
11	12	Shreya	Computer Sc	45	78	66	Pass
12	13	jitendar	Economics	78	30	50	Fail
13	14	Vinit	Math's	65	68	70	Pass
14	15	Anuj	Physics	87	50	60	Pass

#19. To evaluate percentage and display it in a separate column

```
import pandas as pd
```

```
STUDENT_DATA = pd.read_csv('/content/drive/MyDrive/StudentDataSet2_LAB2.csv')
```

```
Percentage = (STUDENT_DATA['General Sc.']+STUDENT_DATA['Computer']+STUDENT_DATA['Maths'])/3
```

```
STUDENT_DATA['Percentage']=Percentage
print(STUDENT_DATA)
```

	Roll	Student	Department	General Sc.	Maths	Computer	Remarks	\
0	1	Agarwal V	Computer Sc	78	80	66	Pass	
1	2	Agnew P	Computer Sc	45	78	66	Pass	
2	3	Ahmad H	Statistics	78	30	50	Fail	
3	4	Alambritis G	Physics	65	68	70	Pass	
4	5	Allen J	Economics	87	50	60	Pass	
5	6	Anthony A	Math's	54	30	30	Fail	
6	7	Antoniou V	Computer Sc	76	70	70	?	
7	8	Archy J	Economics	76	40	40	Fail	
8	9	Armstrong B	Math's	86	30	40	?	
9	10	Arvanitakis I	Physics	68	70	80	Pass	

	Percentage
0	74.666667
1	63.000000
2	52.666667
3	67.666667
4	65.666667
5	38.000000
6	72.000000
7	52.000000
8	52.000000
9	72.666667

✓ 0s completed at 10:40




```
#creating a 3D matrix (vector) using array()
import numpy as np
arrA=np.array([[ -5,2, -3],[6,0,2],[2,6,1]])
arrB=np.array([[2,4,5],[-8,10,3],[-2,-3,-9]])
print(arrA)
print(arrB)
```

```
[[ -5  2 -3]
 [  6  0  2]
 [  2  6  1]]
[[  2  4  5]
 [-8 10  3]
 [-2 -3 -9]]
```

```
#determine the DETERMINANT of the matrix
import numpy as np
arrA=np.array([[ -5,1, -3],[6,0,2],[2,6,1]])
arrB=np.array([[2,4,5],[-8,10,3],[-2,-3,-9]])
print("Matrix A is\n",arrA)
print("Matrix B is\n",arrB)
DETA=np.linalg.det(arrA)
DETB=np.linalg.det(arrB)
print("Determinant of the matrix A is",DETA)
print("Determinant of the matrix B is",DETB)
```

```
↳ Matrix A is
[[ -5  1 -3]
 [  6  0  2]
 [  2  6  1]]
Matrix B is
[[  2  4  5]
 [-8 10  3]
 [-2 -3 -9]]
Determinant of the matrix A is -49.99999999999999
Determinant of the matrix B is -254.0
```

```
#determine the rank of the matrix
import numpy as np
arr1=np.array([[0,0,0,0],[4,2,3,0],[1,0,0,0],[4,0,3,0]])
arr2=np.array([[5,4,7],[5,-6,5],[4,2,-3]])
arr3=np.array([[1,0,1],[2,1,0],[0,-1,2]])
arr4=np.array([[1,1,1],[3,4,-1],[-1,-2,5]])
arr5=np.array([[1,3,4],[3,9,12],[1,3,4]])
print("Matrix 1 is\n",arr1)
print("Matrix 2 is\n",arr2)
print("Matrix 3 is\n",arr3)
print("Matrix 4 is\n",arr4)
print("Matrix 5 is\n",arr5)
rank1=np.linalg.matrix_rank(arr1)
rank2=np.linalg.matrix_rank(arr2)
rank3=np.linalg.matrix_rank(arr3)
rank4=np.linalg.matrix_rank(arr4)
RANK5=np.linalg.matrix_rank(arr5)
```

```

print("Rank of the 1st matrix is",rank1)
print("Rank of the 2nd matrix is",rank2)
print("Rank of the 3rd matrix is",rank3)
print("Rank of the 4th matrix is",rank4)
print("Rank of the 5th matrix is",RANK5)

```

Matrix 1 is

```

[[0 0 0 0]
 [4 2 3 0]
 [1 0 0 0]
 [4 0 3 0]]

```

Matrix 2 is

```

[[ 5  4  7]
 [ 5 -6  5]
 [ 4  2 -3]]

```

Matrix 3 is

```

[[ 1  0  1]
 [ 2  1  0]
 [ 0 -1  2]]

```

Matrix 4 is

```

[[ 1  1  1]
 [ 3  4 -1]
 [-1 -2  5]]

```

Matrix 5 is

```

[[ 1  3  4]
 [ 3  9 12]
 [ 1  3  4]]

```

Rank of the 1st matrix is 3

Rank of the 2nd matrix is 3

Rank of the 3rd matrix is 2

Rank of the 4th matrix is 3

Rank of the 5th matrix is 1

#TO ADD AND SUB TWO MATRICES

```
import numpy as np
```

```
arrA=np.array([[ -5,1, -3],[6,0,2],[2,6,1]])
```

```
arrB=np.array([[2,4,5],[-8,10,3],[-2,-3,-9]])
```

```
print(arrA+arrB)
```

```
print(arrA-arrB)
```

```

[[-3  5  2]
 [-2 10  5]
 [ 0  3 -8]]
[[-7  -3 -8]
 [14 -10 -1]
 [ 4  9 10]]

```

#TO multiply TWO MATRICES

```
import numpy as np
```

```
arrA=np.array([[ -5,1, -3],[6,0,2],[2,6,1]])
```

```
arrB=np.array([[2,4,5],[8,10,3],[-2,-3,-9]])
```

```
arrC=np.dot(arrA,arrB)
```

```
print(arrC)
```

```

[[ 4 -1  5]
 [ 8 18 12]
 [50 65 19]]

```

```

#determine the nullity of the matrix
import numpy as np
arr1=np.array([[0,0,0,0],[4,2,3,0],[1,0,0,0],[4,0,3,0]])
arr2=np.array([[5,4,7],[5,-6,5],[4,2,-3]])
arr3=np.array([[1,0,1],[2,1,0],[0,-1,2]])
arr4=np.array([[1,1,1],[3,4,-1],[-1,-2,5]])
arr5=np.array([[1,3,4],[3,9,12],[1,3,4]])
print(" Matrix 1 is\n",arr1)
print("Matrix 2 is\n",arr2)
print("Matrix 3 is\n",arr3)
print("Matrix 4 is\n",arr4)
print("Matrix 5 is\n",arr5)
rank1=np.linalg.matrix_rank(arr1)
column1=arr1.shape[1]
rank2=np.linalg.matrix_rank(arr2)
column2=arr2.shape[1]
rank3=np.linalg.matrix_rank(arr3)
column3=arr3.shape[1]
rank4=np.linalg.matrix_rank(arr4)
column4=arr4.shape[1]
RANK5=np.linalg.matrix_rank(arr5)
column5=arr5.shape[1]
print("Rank of the 1st matrix is",rank1)
print("Nullity of the 1st matrix is",column1-rank1)
print("Rank of the 2nd matrix is",rank2)
print("Nullity of the 2nd matrix is",column2-rank2)
print("Rank of the 3rd matrix is",rank3)
print("Nullity of the 3rd matrix is",column3-rank3)
print("Rank of the 4th matrix is",rank4)
print("Nullity of the 4th matrix is",column4-rank4)
print("Rank of the 5th matrix is",RANK5)
print("Nullity of the 5th matrix is",column5-RANK5)

```

```

Matrix 1 is
[[0 0 0 0]
 [4 2 3 0]
 [1 0 0 0]
 [4 0 3 0]]

```

```

Matrix 2 is
[[ 5  4  7]
 [ 5 -6  5]
 [ 4  2 -3]]

```

```

Matrix 3 is
[[ 1  0  1]
 [ 2  1  0]
 [ 0 -1  2]]

```

```

Matrix 4 is
[[ 1  1  1]
 [ 3  4 -1]
 [-1 -2  5]]

```

```

Matrix 5 is
[[ 1  3  4]
 [ 3  9 12]
 [ 1  3  4]]

```

```

Rank of the 1st matrix is 3

```

```
Nullity of the 1st matrix is 1  
Rank of the 2nd matrix is 3  
Nullity of the 2nd matrix is 0  
Rank of the 3rd matrix is 2  
Nullity of the 3rd matrix is 1  
Rank of the 4th matrix is 3  
Nullity of the 4th matrix is 0  
Rank of the 5th matrix is 1  
Nullity of the 5th matrix is 2
```

[Colab paid products](#) - [Cancel contracts here](#)



```
# 1- Check if u or v is a unit vector.
```

```
import numpy as np
```

```
U=np.array([0.5, 0.4, 0.4, 0.5, 0.1, 0.4, 0.1])
magU = np.linalg.norm(U)
unit_U = U / magU
print("Unit vector u:\n",unit_U,"\n")
```

```
Unit vector u:
[0.5 0.4 0.4 0.5 0.1 0.4 0.1]
```

```
# 1- Check if u or v is a unit vector
```

```
import numpy as np
```

```
V=np.array([-1,-2, 1,-2, 3, 1,-5])
magV = np.linalg.norm(V)
unit_V = V / magV
print("Unit vector v:\n",unit_V)
```

```
Unit vector v:
[-0.1490712  -0.2981424   0.1490712  -0.2981424   0.4472136   0.1490712]
```

Saved successfully!



```
# 1- Calculate the dot product, <u, v>
```

```
import numpy as np
```

```
U=np.array([0.5, 0.4, 0.4, 0.5, 0.1, 0.4, 0.1])
V=np.array([-1,-2, 1,-2, 3, 1,-5])
```

```
UdotV= np.dot(U,V)
print("U*V = ", UdotV)
```

```
U*V = -1.7
```

```
# 1- Are u and v orthogonal ?
```

```
import numpy as np
```

```
U=np.array([0.5, 0.4, 0.4, 0.5, 0.1, 0.4, 0.1])
V=np.array([-1,-2, 1,-2, 3, 1,-5])
```

```
UdotV= np.dot(U,V)
print("U*V= ", UdotV)
```

```
if (UdotV==0):
    print("orthogonal")
else:
    print("not orthogonal")
```

```
U*V= -1.7
not orthogonal
```

```
# 2- Evaluate dot product <v,w>
```

```
import numpy as np
```

```
V = (-4, 3, -2, 2, 1, -3, 4, 1, -2)
W = (3,3,3,-1,6,-1,2,-5,-7)
```

```
VdotW= np.dot(V,W)
print("U*V = ", VdotW)
```

```
U dot V = 15
```

```
# 2- Are any pair of vectors orthogonal, and if so which ones?
```

```
import numpy as np
```

```

U = (1, 2, 5, 2,-3, 1, 2, 6, 2)
V = (-4, 3, -2, 2, 1, -3, 4, 1, -2)
W = (3,3,3,-1,6,-1,2,-5,-7)

```

```

UdotV= np.dot(U,V)
print("U*V = ", UdotV)

```

```

if (UdotV==0):
    print("U*V are orthogonal")
else:
    print("U*V are not orthogonal")

```

```

VdotW= np.dot(V,W)
print("\n", "V*W = ", VdotW)

```

```

if (VdotW==0):
    print("V*W are orthogonal")
else:
    print("V*W are not orthogonal")
UdotW= np.dot(U,W)
print("\n", "U*W= ", UdotW)

```

```

if (UdotW==0):
    print("U*W are orthogonal")
else:
    print("U*W are not orthogonal")

```

Saved successfully!

U*V are orthogonal

V*W = 15
V*W are not orthogonal

U*W= -37
U*W are not orthogonal

```

# 3- A^T B
import numpy as np

```

```

A = np.array([[2,-2], [-3,1], [5,-3]])
B = np.array([[4,4,4], [-2,3,-7], [2,5,-7]])
print("Matrix A:\n",A,"\n",)
A_T = A.transpose()
print("Transpose of A:\n",A_T,"\n")
print("Matrix B:\n",B,"\n",)

```

```

A_TdotB= np.dot(A_T,B)
print("A^T *B is:\n",A_TdotB)

```

Matrix A:
[[2 -2]
[-3 1]
[5 -3]]

Transpose of A:
[[2 -3 5]
[-2 1 -3]]

Matrix B:
[[4 4 4]
[-2 3 -7]
[2 5 -7]]

A^T *B is:
[[24 24 -6]
[-16 -20 6]]

```

# 3- C + B

```

```

import numpy as np

```

```

B = np.array([[4,4,4], [-2,3,-7], [2,5,-7]])
C = np.array([[4,-1,2], [-8,2,-4], [2,1,-4]])
print("Matrix B:\n",B)
print("\n Matrix C:\n",C)
print("\n B+C Matrix:\n",B+C)

```

```
Matrix B:
[[ 4  4  4]
 [-2  3 -7]
 [ 2  5 -7]]
```

```
Matrix C:
[[ 4 -1  2]
 [-8  2 -4]
 [ 2  1 -4]]
```

```
B+C Matrix:
[[ 8  3  6]
 [-10  5 -11]
 [ 4  6 -11]]
```

3- Which matrices are full rank?

```
import numpy as np
```

```
A = np.array([[2,-2], [-3,1], [5,-3]])
B = np.array([[4,4,4], [-2,3,-7], [2,5,-7]])
C = np.array([[4,-1,2], [-8,2,-4], [2,1,-4]])
```

```
ROWS1=A.shape[0]
COLUMN1=A.shape[1]
if (ROWS1==COLUMN1):
    print("A is full rank")
```

Saved successfully!

```
ROWS2=B.shape[0]
COLUMN2=B.shape[1]
if (ROWS2==COLUMN2):
    print("B is full rank")
else:
    print("B is not full rank")
```

```
ROWS3=C.shape[0]
COLUMN3=C.shape[1]
if (ROWS3==COLUMN3):
    print("C is a full rank")
else:
    print("C is not full rank")
```

```
A is not full rank
B is full rank
C is a full rank
```

3- B^{-1}

```
import numpy as np
```

```
B = np.array([[4,4,4], [-2,3,-7], [2,5,-7]])
B_Inverse= np.linalg.inv(B)
print(B_Inverse)
```

```
[[-0.11666667 -0.4      0.33333333]
 [ 0.23333333  0.3     -0.16666667]
 [ 0.13333333  0.1     -0.16666667]]
```



Saved successfully! 


```
#A.1.display the column names and the number of records/column
import pandas as pd
Employee_data=pd.read_csv("/content/EMP-DataSet.csv")
print(Employee_data.info())
COL=Employee_data.columns
for x in COL:
    print(x)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0    EMP ID      8 non-null      int64
1    EMP NAME    8 non-null      object
2    SALARY      6 non-null      float64
3    START DATE  8 non-null      object
dtypes: float64(1), int64(1), object(2)
memory usage: 384.0+ bytes
None
EMP ID
EMP NAME
SALARY
START DATE
```

```
2.#display the first 4 records of the database
import pandas as pd
Employee_data=pd.read_csv("/content/EMP-DataSet.csv")
print(Employee_data.head(4))
```

	EMP ID	EMP NAME	SALARY	START DATE
0	1	Satish	50000.0	01-11-2017
1	2	Reeya	75000.0	12-05-2016
2	3	Jay	100000.0	22-09-2015
3	4	Rahul	NaN	11-10-2016

```
3.#evaluate various statistical parameters using described () function
import pandas as pd
Employee_data=pd.read_csv("/content/EMP-DataSet.csv")
print(Employee_data.describe())
```

	EMP ID	SALARY
count	8.00000	6.000000
mean	4.50000	70833.333333
std	2.44949	24782.386218
min	1.00000	45000.000000
25%	2.75000	51250.000000
50%	4.50000	65000.000000
75%	6.25000	93750.000000
max	8.00000	100000.000000

```
4.#check for the missing values in the dataset
import pandas as pd
```

```
Employee_data=pd.read_csv("/content/EMP-DataSet.csv")
Employee_data.isnull()
```

	EMP ID	EMP NAME	SALARY	START DATE
0	False	False	False	False
1	False	False	False	False
2	False	False	False	False
3	False	False	True	False
4	False	False	False	False
5	False	False	False	False
6	False	False	True	False
7	False	False	False	False

```
5.#check for the missing values in the dataset and replace them with some valid numeric va
import pandas as pd
Employee_data=pd.read_csv("/content/EMP-DataSet.csv")
Employee_data['SALARY'].fillna(40000,inplace=True)
print(Employee_data)
```

	EMP ID	EMP NAME	SALARY	START DATE
0	1	Satish	50000.0	01-11-2017
1	2	Reeya	75000.0	12-05-2016
2	3	Jay	100000.0	22-09-2015
3	4	Rahul	40000.0	11-10-2016
4	5	Roy	45000.0	08-01-2017
5	6	Jay	100000.0	22-09-2015
6	7	Vishal	40000.0	05-01-2016
7	8	Serah	55000.0	06-02-2018

```
6.#find duplicate recordsin the dataset
import pandas as pd
Employee_data=pd.read_csv("/content/EMP-DataSet.csv")
print(Employee_data.duplicated())
```

```
0    False
1    False
2    False
3    False
4    False
5    False
6    False
7    False
dtype: bool
```

```
#THERE IS NO DUPLICATE VALUE IN THE DATASET
```

```
#B.1.Number of rows and column and column name
import pandas as pd
Patient_data=pd.read_csv("/content/diabetes.csv")
print(Patient_data.info())
COL=Patient_data.columns
for x in COL:
    print(x)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Pregnancies            768 non-null   int64
1   Glucose                768 non-null   int64
2   BloodPressure          768 non-null   int64
3   SkinThickness          768 non-null   int64
4   Insulin                768 non-null   int64
5   BMI                    768 non-null   float64
6   DiabetesPedigreeFunction 768 non-null   float64
7   Age                    768 non-null   int64
8   Outcome                768 non-null   int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome
```

```
2.#Display first 10 records of the dataset
import pandas as pd
Patient_data=pd.read_csv("/content/diabetes.csv")
print(Patient_data.head(10))
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	
5	5	116	74	0	0	25.6	
6	3	78	50	32	88	31.0	
7	10	115	0	0	0	35.3	
8	2	197	70	45	543	30.5	
9	8	125	96	0	0	0.0	

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0

2	0.672	32	1
3	0.167	21	0
4	2.288	33	1
5	0.201	30	0
6	0.248	26	1
7	0.134	29	0
8	0.158	53	1
9	0.232	54	1

3.#statistical parameters using describe () function

```
import pandas as pd
```

```
Patient_data=pd.read_csv("/content/diabetes.csv")
```

```
print(Patient_data.describe())
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin \
count	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479
std	3.369578	31.972618	19.355807	15.952218	115.244002
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000
75%	6.000000	140.250000	80.000000	32.000000	127.250000
max	17.000000	199.000000	122.000000	99.000000	846.000000

	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000
mean	31.992578	0.471876	33.240885	0.348958
std	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.078000	21.000000	0.000000
25%	27.300000	0.243750	24.000000	0.000000
50%	32.000000	0.372500	29.000000	0.000000
75%	36.600000	0.626250	41.000000	1.000000
max	67.100000	2.420000	81.000000	1.000000

4.#Finding and removing duplicate values

```
import pandas as pd
```

```
Patient_data=pd.read_csv("/content/diabetes.csv")
```

```
print(Patient_data.duplicated())
```

```

0      False
1      False
2      False
3      False
4      False
...
763    False
764    False
765    False
766    False
767    False
Length: 768, dtype: bool

```

#IT DOES NOT CONTAIN ANY DUPLICATE VALUE

5.#presence of missing value in the dataset

```
import pandas as pd
Patient_data=pd.read_csv("/content/diabetes.csv")
Patient_data.isnull()
```

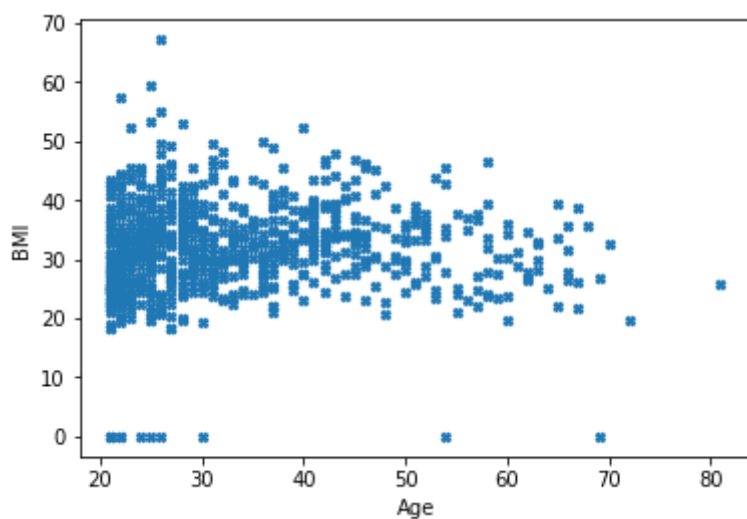
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedi
0	False	False	False	False	False	False	
1	False	False	False	False	False	False	
2	False	False	False	False	False	False	
3	False	False	False	False	False	False	
4	False	False	False	False	False	False	
...	...	...	...	...	...	...	
763	False	False	False	False	False	False	
764	False	False	False	False	False	False	
765	False	False	False	False	False	False	
766	False	False	False	False	False	False	
767	False	False	False	False	False	False	

768 rows × 9 columns

IT DOES NOT CONTAIN ANY NULL VALUE

6.# show scatter plot depicting relationship between two numeric columns of your choice

```
import pandas as pd
import matplotlib.pyplot as plt
Patient_data=pd.read_csv("/content/diabetes.csv")
Patient_data.plot(kind='scatter',x='Age',y='BMI',marker='X')
plt.show()
```



[Colab paid products](#) - [Cancel contracts here](#)



```
import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')

print(Calories)
```

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
18	45	'2020/12/18'	90	112	NaN
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
22	45	NaN	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	20201226	100	120	250.0
27	60	'2020/12/27'	92	118	241.0
28	60	'2020/12/28'	103	132	NaN
29	60	'2020/12/29'	100	132	280.0
30	60	'2020/12/30'	102	129	380.3
31	60	'2020/12/31'	92	115	243.0

```
import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')
print(Calories.count())
```

```
Duration    32
Date        31
Pulse       32
Maxpulse    32
Calories    30
```

```
dtype: int64
```

```
Duration    68.4375
Pulse       103.5000
Maxpulse    128.5000
Calories    304.6800
```

```
dtype: float64
```

```
<ipython-input-2-6739750f6ea4>:4: FutureWarning: Dropping of nuisance columns in Data
print(Calories.mean())
```

```
import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')
print(Calories.mean())
```

Duration 68.4375
Pulse 103.5000
Maxpulse 128.5000
Calories 304.6800
dtype: float64
<ipython-input-3-e00fdfab0700>:3: FutureWarning: Dropping of nuisance columns in Data
print(Calories.mean())

```
import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')
print(Calories.min())
```

Duration 30.0
Pulse 90.0
Maxpulse 101.0
Calories 195.1
dtype: float64
<ipython-input-4-df28aaf766f9>:3: FutureWarning: Dropping of nuisance columns in Data
print(Calories.min())

```
import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')
print(Calories.max())
```

Duration 450.0
Pulse 130.0
Maxpulse 175.0
Calories 479.0
dtype: float64
<ipython-input-5-aadf9c5a25fb>:3: FutureWarning: Dropping of nuisance columns in Data
print(Calories.max())

```
import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')
print(Calories.std())
```

Duration 70.039591
Pulse 7.832933
Maxpulse 12.998759
Calories 66.003779
dtype: float64
<ipython-input-6-110ddade851c>:3: FutureWarning: Dropping of nuisance columns in Data
print(Calories.std())


```
import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')
print(Calories.mode())
```

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/12'	100	120	300.0

```
import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')
print(Calories.median())
```

```
Duration      60.0
Pulse         102.5
Maxpulse      127.5
Calories      291.2
dtype: float64
<ipython-input-9-314e48cc02c8>:3: FutureWarning: Dropping of nuisance columns in Data
print(Calories.median())
```

```
import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')
print(Calories.describe())
```

	Duration	Pulse	Maxpulse	Calories
count	32.000000	32.000000	32.000000	30.000000
mean	68.437500	103.500000	128.500000	304.680000
std	70.039591	7.832933	12.998759	66.003779
min	30.000000	90.000000	101.000000	195.100000
25%	60.000000	100.000000	120.000000	250.700000
50%	60.000000	102.500000	127.500000	291.200000
75%	60.000000	106.500000	132.250000	343.975000
max	450.000000	130.000000	175.000000	479.000000

```
import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')
Calories_clean = Calories.dropna()
print(Calories_clean)
```

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7

12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	20201226	100	120	250.0
27	60	'2020/12/27'	92	118	241.0
29	60	'2020/12/29'	100	132	280.0
30	60	'2020/12/30'	102	129	380.3
31	60	'2020/12/31'	92	115	243.0

```
import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')
Calories["Date"].fillna("'2020/12/31'",inplace=True)
Calories["Calories"].fillna(100,inplace=True)
print(Calories)
```

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
18	45	'2020/12/18'	90	112	100.0
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
22	45	'2020/12/31'	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	20201226	100	120	250.0
27	60	'2020/12/27'	92	118	241.0
28	60	'2020/12/28'	103	132	100.0
29	60	'2020/12/29'	100	132	280.0
30	60	'2020/12/30'	102	129	380.3
31	60	'2020/12/31'	92	115	243.0

```
import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')
Calories_clean = Calories.isnull()
print(Calories_clean)
```

	Duration	Date	Pulse	Maxpulse	Calories
0	False	False	False	False	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False
5	False	False	False	False	False
6	False	False	False	False	False
7	False	False	False	False	False
8	False	False	False	False	False
9	False	False	False	False	False
10	False	False	False	False	False
11	False	False	False	False	False
12	False	False	False	False	False
13	False	False	False	False	False
14	False	False	False	False	False
15	False	False	False	False	False
16	False	False	False	False	False
17	False	False	False	False	False
18	False	False	False	False	True
19	False	False	False	False	False
20	False	False	False	False	False
21	False	False	False	False	False
22	False	True	False	False	False
23	False	False	False	False	False
24	False	False	False	False	False
25	False	False	False	False	False
26	False	False	False	False	False
27	False	False	False	False	False
28	False	False	False	False	True
29	False	False	False	False	False
30	False	False	False	False	False
31	False	False	False	False	False

```
import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')
print(Calories_clean)
Calories_clean = Calories.duplicated().sum()
print("No of duplicate entries:",Calories_clean)
Calories_clean = Calories.drop_duplicates()
```

9	False
10	False
11	False
12	True
13	False
14	False
15	False
16	False
17	False
18	False

```

19 False
20 False
21 False
22 False
23 False
24 False
25 False
26 False
27 False
28 False
29 False
30 False
31 False

```

```
dtype: bool
```

```
No of duplicate entries: 1
```

```

No of duplicate entries: Duration Date Pulse Maxpulse Calories
0      60 '2020/12/01' 110      130    409.1
1      60 '2020/12/02' 117      145    479.0
2      60 '2020/12/03' 103      135    340.0
3      45 '2020/12/04' 109      175    282.4
4      45 '2020/12/05' 117      148    406.0
5      60 '2020/12/06' 102      127    300.0
6      60 '2020/12/07' 110      136    374.0
7     450 '2020/12/08' 104      134    253.3
8      30 '2020/12/09' 109      133    195.1
9      60 '2020/12/10'  98      124    269.0
10     60 '2020/12/11' 103      147    329.3
11     60 '2020/12/12' 100      120    250.7
13     60 '2020/12/13' 106      128    345.3
14     60 '2020/12/14' 104      132    379.3
15     60 '2020/12/15'  98      123    275.0
16     60 '2020/12/16'  98      120    215.2
17     60 '2020/12/17' 100      120    300.0
18     45 '2020/12/18'  90      112      NaN
19     60 '2020/12/19' 103      123    323.0
20     45 '2020/12/20'  97      125    243.0
21     60 '2020/12/21' 108      131    364.2
22     45      NaN    100      119    282.0
23     60 '2020/12/23' 130      101    300.0
24     45 '2020/12/24' 105      132    246.0
25     60 '2020/12/25' 102      126    334.5
26     60    20201226    100      120    250.0
27     60 '2020/12/27'  92      118    241.0
28     60 '2020/12/28' 103      132      NaN
29     60 '2020/12/29' 100      132    280.0
30     60 '2020/12/30' 102      129    380.3
31     60 '2020/12/31'  92      115    243.0

```

```

import pandas as pd
Calories = pd.read_csv('CaloriesDataSet.csv')
Calories_clean = Calories.duplicated()
print(Calories_clean)
Calories_clean = Calories.corr()
print("No of duplicate entries:",Calories_clean)

```

```

0 False
1 False
2 False
3 False

```

```

4      False
5      False
6      False
7      False
8      False
9      False
10     False
11     False
12      True
13     False
14     False
15     False
16     False
17     False
18     False
19     False
20     False
21     False
22     False
23     False
24     False
25     False
26     False
27     False
28     False
29     False
30     False
31     False
dtype: bool

```

```

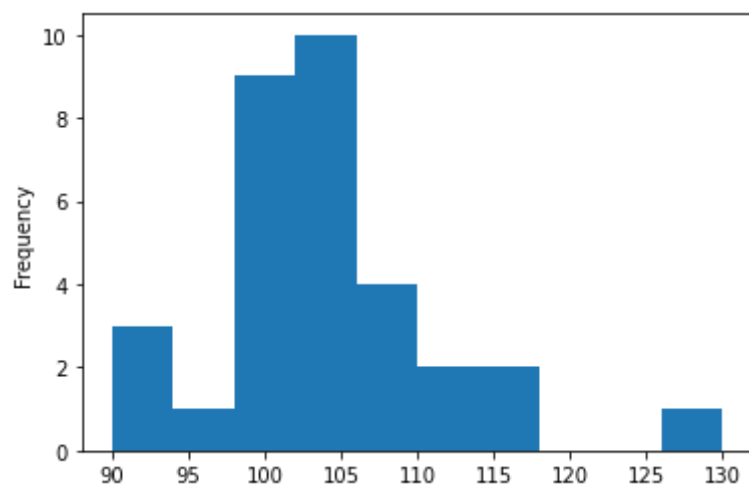
No of duplicate entries:      Duration      Pulse  Maxpulse  Calories
Duration  1.000000  0.004410  0.049959 -0.114169
Pulse      0.004410  1.000000  0.276583  0.513186
Maxpulse   0.049959  0.276583  1.000000  0.357460
Calories   -0.114169  0.513186  0.357460  1.000000

```

```

import matplotlib.pyplot as plt
import seaborn as sns
Calories["Pulse"].plot(kind="hist")
plt.show()

```

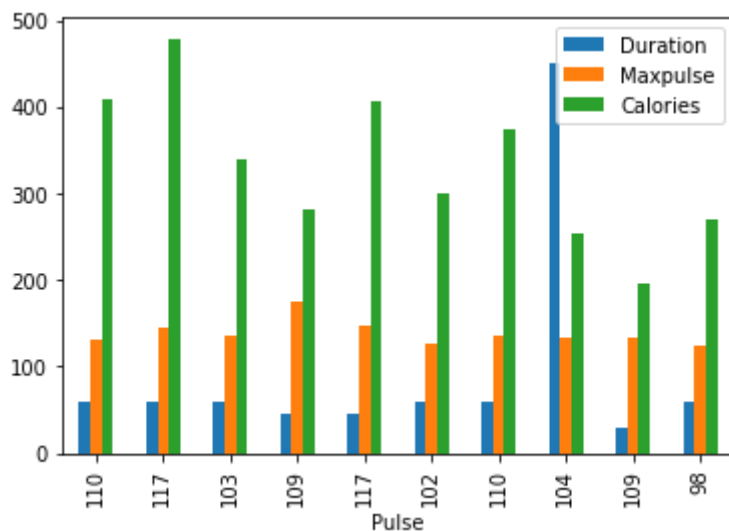


```

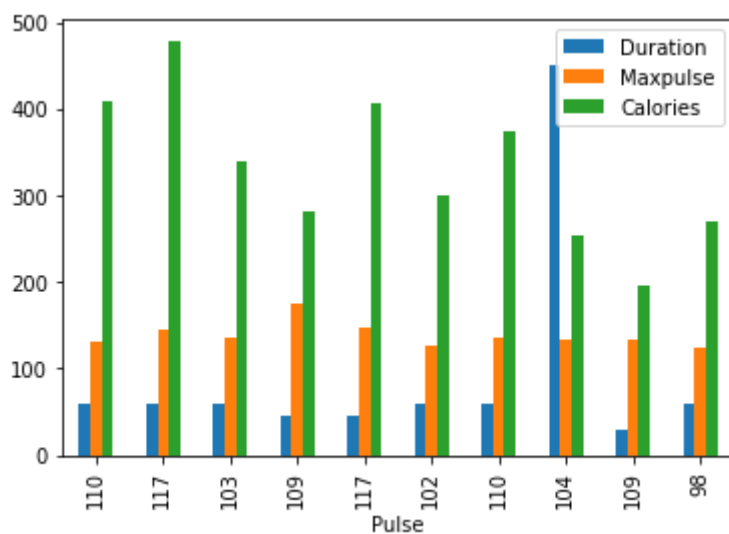
import matplotlib.pyplot as plt
import seaborn as sns

```

```
Calories.head(10).plot(kind='bar',x='Pulse')
plt.show()
```

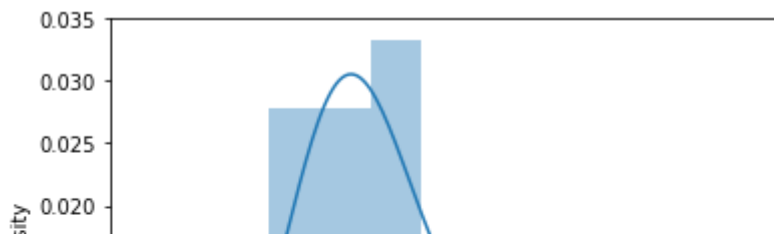


```
import matplotlib.pyplot as plt
import seaborn as sns
Calories = pd.read_csv('CaloriesDataSet.csv')
Calories.head(10).plot(kind='bar',x='Pulse')
plt.show()
```

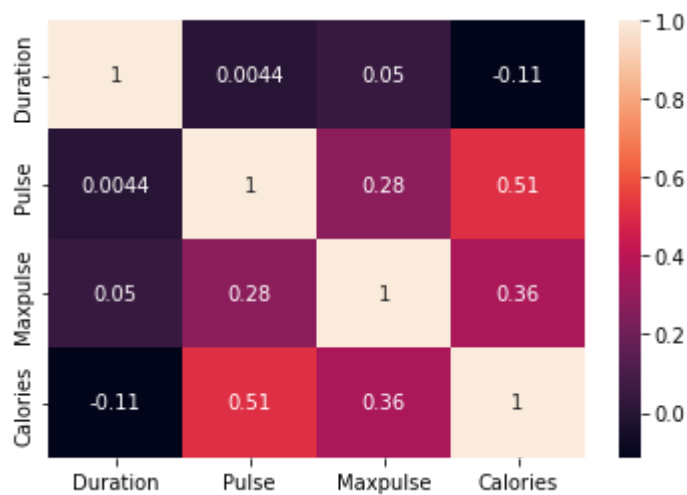


```
import matplotlib.pyplot as plt
import seaborn as sns
sns.distplot(Calories['Maxpulse'].head(20))
plt.show()
```

```
/usr/local/lib/python3.8/dist-packages/seaborn/distributions.py:2619: FutureWarning:
warnings.warn(msg, FutureWarning)
```



```
import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(Calories.corr(),annot=True)
plt.show()
```



```
import matplotlib.pyplot as plt
import seaborn as sns
Calories.agg(['sum','min','max'])
```

	Duration	Pulse	Maxpulse	Calories
sum	2190	3312	4112	9140.4
min	30	90	101	195.1
max	450	130	175	479.0

```
import matplotlib.pyplot as plt
import seaborn as sns
Calories.groupby('Duration').agg(['sum','mean','median','min','max','prod'])
```



```

MEDV      506
dtype: int64

```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	\
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1	296	
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2	242	
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2	242	
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3	222	
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3	222	
..	...	...	...	...	...	...	...	...	...	...	
501	0.06263	0.0	11.93	0.0	0.573	6.593	69.1	2.4786	1	273	
502	0.04527	0.0	11.93	0.0	0.573	6.120	76.7	2.2875	1	273	
503	0.06076	0.0	11.93	0.0	0.573	6.976	91.0	2.1675	1	273	
504	0.10959	0.0	11.93	0.0	0.573	6.794	89.3	2.3889	1	273	
505	0.04741	0.0	11.93	0.0	0.573	6.030	NaN	2.5050	1	273	

```

PTRATIO    B  LSTAT  MEDV
0      15.3 396.90   4.98 24.0
1      17.8 396.90   9.14 21.6
2      17.8 392.83   4.03 34.7
3      18.7 394.63   2.94 33.4
4      18.7 396.90   NaN 36.2
..      ...   ...   ...   ...
501     21.0 391.99   NaN 22.4
502     21.0 396.90   9.08 20.6
503     21.0 396.90   5.64 23.9
504     21.0 393.45   6.48 22.0
505     21.0 396.90   7.88 11.9

```

[506 rows x 14 columns]

```

import pandas as pd
Data = pd.read_csv('HousingPrice-DataSet.csv')
print(Data.mean())

```

```

CRIM      3.611874
ZN        11.211934
INDUS     11.083992
CHAS      0.069959
NOX       0.554695
RM        6.284634
AGE       68.518519
DIS       3.795043
RAD       9.549407
TAX      408.237154
PTRATIO   18.455534
B        356.674032
LSTAT    12.715432
MEDV     22.532806
dtype: float64

```

```

import pandas as pd
Data = pd.read_csv('HousingPrice-DataSet.csv')
print(Data.min())

```

```

CRIM      0.00632
ZN        0.00000
INDUS     0.46000

```

```

CHAS      0.00000
NOX       0.38500
RM        3.56100
AGE       2.90000
DIS       1.12960
RAD       1.00000
TAX      187.00000
PTRATIO   12.60000
B         0.32000
LSTAT     1.73000
MEDV      5.00000
dtype: float64

```

```

import pandas as pd
Data = pd.read_csv('HousingPrice-DataSet.csv')
print(Data.mode())

```

```

      CRIM    ZN  INDUS  CHAS    NOX     RM   AGE     DIS  RAD    TAX  \
0   0.01501  0.0   18.1   0.0  0.538  5.713  100.0  3.4952  24.0  666.0
1  14.33370  NaN   NaN   NaN   NaN   6.127   NaN   NaN   NaN   NaN
2     NaN   NaN   NaN   NaN   NaN   6.167   NaN   NaN   NaN   NaN
3     NaN   NaN   NaN   NaN   NaN   6.229   NaN   NaN   NaN   NaN
4     NaN   NaN   NaN   NaN   NaN   6.405   NaN   NaN   NaN   NaN
5     NaN   NaN   NaN   NaN   NaN   6.417   NaN   NaN   NaN   NaN

      PTRATIO    B  LSTAT  MEDV
0     20.2  396.9   6.36  50.0
1     NaN   NaN   7.79   NaN
2     NaN   NaN   8.05   NaN
3     NaN   NaN  14.10   NaN
4     NaN   NaN  18.13   NaN
5     NaN   NaN   NaN   NaN

```

```

import pandas as pd
Data = pd.read_csv('HousingPrice-DataSet.csv')
print(Data.median())

```

```

CRIM      0.253715
ZN        0.000000
INDUS     9.690000
CHAS      0.000000
NOX       0.538000
RM        6.208500
AGE       76.800000
DIS       3.207450
RAD       5.000000
TAX      330.000000
PTRATIO   19.050000
B        391.440000
LSTAT     11.430000
MEDV     21.200000
dtype: float64

```

```

import pandas as pd
Data = pd.read_csv('HousingPrice-DataSet.csv')
print(Data.std())

```

```

CRIM      8.720192
ZN        23.388876
INDUS     6.835896
CHAS      0.255340
NOX       0.115878
RM        0.702617
AGE       27.999513
DIS       2.105710
RAD       8.707259
TAX       168.537116
PTRATIO   2.164946
B         91.294864
LSTAT     7.155871
MEDV     9.197104
dtype: float64

```

```

import pandas as pd
Data = pd.read_csv('HousingPrice-DataSet.csv')
print(Data.describe())

```

	CRIM	ZN	INDUS	CHAS	NOX	RM \
count	486.000000	486.000000	486.000000	486.000000	506.000000	506.000000
mean	3.611874	11.211934	11.083992	0.069959	0.554695	6.284634
std	8.720192	23.388876	6.835896	0.255340	0.115878	0.702617
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000
25%	0.081900	0.000000	5.190000	0.000000	0.449000	5.885500
50%	0.253715	0.000000	9.690000	0.000000	0.538000	6.208500
75%	3.560263	12.500000	18.100000	0.000000	0.624000	6.623500
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000

	AGE	DIS	RAD	TAX	PTRATIO	B \
count	486.000000	506.000000	506.000000	506.000000	506.000000	506.000000
mean	68.518519	3.795043	9.549407	408.237154	18.455534	356.674032
std	27.999513	2.105710	8.707259	168.537116	2.164946	91.294864
min	2.900000	1.129600	1.000000	187.000000	12.600000	0.320000
25%	45.175000	2.100175	4.000000	279.000000	17.400000	375.377500
50%	76.800000	3.207450	5.000000	330.000000	19.050000	391.440000
75%	93.975000	5.188425	24.000000	666.000000	20.200000	396.225000
max	100.000000	12.126500	24.000000	711.000000	22.000000	396.900000

	LSTAT	MEDV
count	486.000000	506.000000
mean	12.715432	22.532806
std	7.155871	9.197104
min	1.730000	5.000000
25%	7.125000	17.025000
50%	11.430000	21.200000
75%	16.955000	25.000000
max	37.970000	50.000000

```

import pandas as pd
Data = pd.read_csv('HousingPrice-DataSet.csv')
Data_clean = Data.dropna()
print(Data_clean)

```

CRIM ZN INDUS CHAS NOX RM AGE DIS RAD TAX \

0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1	296
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2	242
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2	242
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3	222
5	0.02985	0.0	2.18	0.0	0.458	6.430	58.7	6.0622	3	222
..	...	...	...	...	...	...	...	...	...	...
499	0.17783	0.0	9.69	0.0	0.585	5.569	73.5	2.3999	6	391
500	0.22438	0.0	9.69	0.0	0.585	6.027	79.7	2.4982	6	391
502	0.04527	0.0	11.93	0.0	0.573	6.120	76.7	2.2875	1	273
503	0.06076	0.0	11.93	0.0	0.573	6.976	91.0	2.1675	1	273
504	0.10959	0.0	11.93	0.0	0.573	6.794	89.3	2.3889	1	273

	PTRATIO	B	LSTAT	MEDV
0	15.3	396.90	4.98	24.0
1	17.8	396.90	9.14	21.6
2	17.8	392.83	4.03	34.7
3	18.7	394.63	2.94	33.4
5	18.7	394.12	5.21	28.7
..	...	...	...	...
499	19.2	395.77	15.10	17.5
500	19.2	396.90	14.33	16.8
502	21.0	396.90	9.08	20.6
503	21.0	396.90	5.64	23.9
504	21.0	393.45	6.48	22.0

[394 rows x 14 columns]

```
import pandas as pd
Data = pd.read_csv('HousingPrice-DataSet.csv')
Data["CRIM"].fillna("0.0",inplace=True)
Data["ZN"].fillna("0.0",inplace=True)
Data["INDUS"].fillna("0.0",inplace=True)
Data["CHAS"].fillna("0.0",inplace=True)
Data["NOX"].fillna("0.0",inplace=True)
Data["RM"].fillna("0.0",inplace=True)
Data["AGE"].fillna("0.0",inplace=True)
Data["RAD"].fillna("0.0",inplace=True)
Data["PTRATIO"].fillna("0.0",inplace=True)
Data["LSTAT"].fillna("0.0",inplace=True)
Data["MEDV"].fillna("0.0",inplace=True)
print(Data)
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	\
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1	296	15.3	
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2	242	17.8	
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2	242	17.8	
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3	222	18.7	
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3	222	18.7	
..	...	...	...	...	...	...	...	...	...	...	...	
501	0.06263	0.0	11.93	0.0	0.573	6.593	69.1	2.4786	1	273	21.0	
502	0.04527	0.0	11.93	0.0	0.573	6.120	76.7	2.2875	1	273	21.0	
503	0.06076	0.0	11.93	0.0	0.573	6.976	91.0	2.1675	1	273	21.0	
504	0.10959	0.0	11.93	0.0	0.573	6.794	89.3	2.3889	1	273	21.0	
505	0.04741	0.0	11.93	0.0	0.573	6.030	0.0	2.5050	1	273	21.0	

B LSTAT MEDV

```

0      396.90  4.98  24.0
1      396.90  9.14  21.6
2      392.83  4.03  34.7
3      394.63  2.94  33.4
4      396.90   0.0  36.2
..      ...   ...   ...
501    391.99   0.0  22.4
502    396.90  9.08  20.6
503    396.90  5.64  23.9
504    393.45  6.48  22.0
505    396.90  7.88  11.9

```

```
[506 rows x 14 columns]
```

```

import pandas as pd
Data_clean = Data.isnull().sum()
print(Data_clean)

```

```

CRIM      0
ZN        0
INDUS     0
CHAS      0
NOX       0
RM        0
AGE       0
DIS       0
RAD       0
TAX       0
PTRATIO   0
B         0
LSTAT     0
MEDV      0
dtype: int64

```

```
import pandas as pd
```

```

Data_clean = Data.duplicated().sum()
print("No of duplicate entries:",Data_clean)
Data_clean = Data.drop_duplicates()

```

```
No of duplicate entries: 0
```

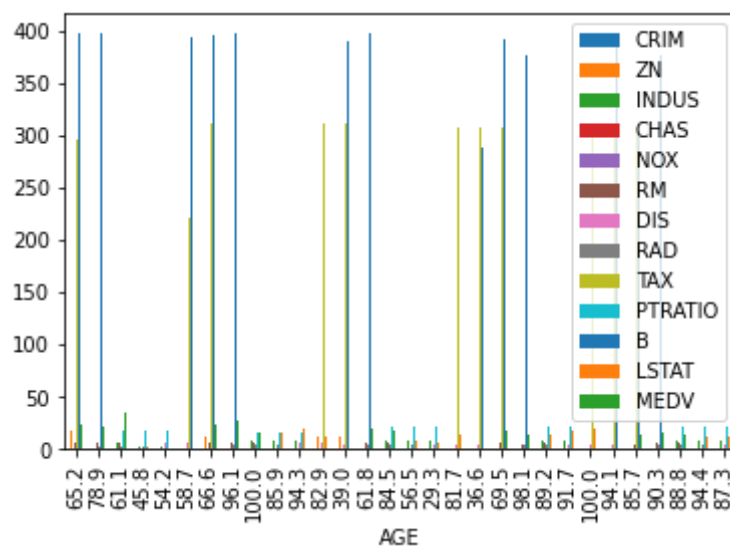
```

import matplotlib.pyplot as plt
import seaborn as sns
Data["TAX"].plot(kind="hist")
plt.show()

```



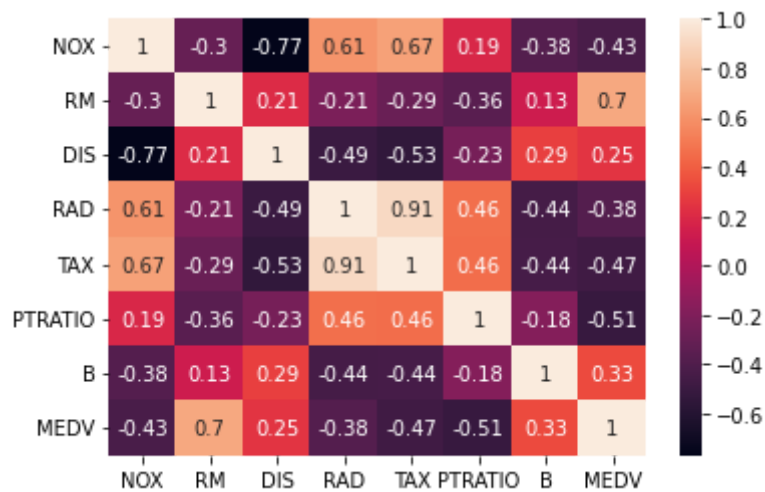
```
import matplotlib.pyplot as plt
import seaborn as sns
Calories = pd.read_csv('HousingPrice-DataSet.csv')
Calories.head(30).plot(kind='bar',x='AGE')
plt.show()
```



```
import matplotlib.pyplot as plt
import seaborn as sns
sns.distplot(Data['TAX'].head(50))
plt.show()
```

/usr/local/lib/python3.8/dist-packages/seaborn/distributions.py:2619: FutureWarning:

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(Data.corr(),annot=True)
plt.show()
```



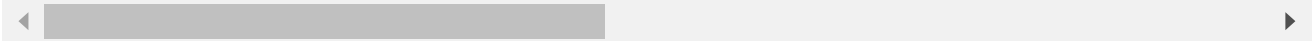
```
import matplotlib.pyplot as plt
import seaborn as sns
Data.agg(['sum','min','max'])
```

	NOX	RM	DIS	RAD	TAX	PTRATIO	B	MEDV
sum	280.6757	3180.025	1920.2916	4832	206568	9338.5	180477.06	11401.6
min	0.3850	3.561	1.1296	1	187	12.6	0.32	5.0
max	0.8710	8.780	12.1265	24	711	22.0	396.90	50.0

```
import matplotlib.pyplot as plt
import seaborn as sns
Calories.groupby('TAX').agg(['sum','mean','median','min','max','prod'])
```

	CRIM						ZN		
	sum	mean	median	min	max	prod	sum	mean	medi
TAX									
187	0.01709	0.017090	0.017090	0.01709	0.01709	1.709000e-02	90.0	90.0	9
188	1.03887	0.148410	0.098490	0.06899	0.38735	4.457204e-07	0.0	0.0	
193	0.48316	0.069023	0.065880	0.05602	0.09103	6.719527e-09	0.0	0.0	
198	0.01501	0.015010	0.015010	0.01501	0.01501	1.501000e-02	90.0	90.0	9
216	0.36401	0.072802	0.037050	0.01951	0.21038	3.334878e-07	97.5	19.5	2
...	...	...	...	...	...	...	...	...	...
432	1.14646	0.143308	0.136945	0.10084	0.22212	1.475769e-07	0.0	0.0	
437	8.84148	0.589432	0.352330	0.24980	1.62864	2.340341e-05	0.0	0.0	
469	0.01360	0.013600	0.013600	0.01360	0.01360	1.360000e-02	75.0	75.0	7
666	1613.39850	13.011278	9.209660	2.37857	88.97620	1.538223e+122	0.0	0.0	
711	0.75875	0.151750	0.150860	0.10574	0.20746	6.755372e-05	0.0	0.0	

66 rows × 78 columns



[Colab paid products](#) - [Cancel contracts here](#)



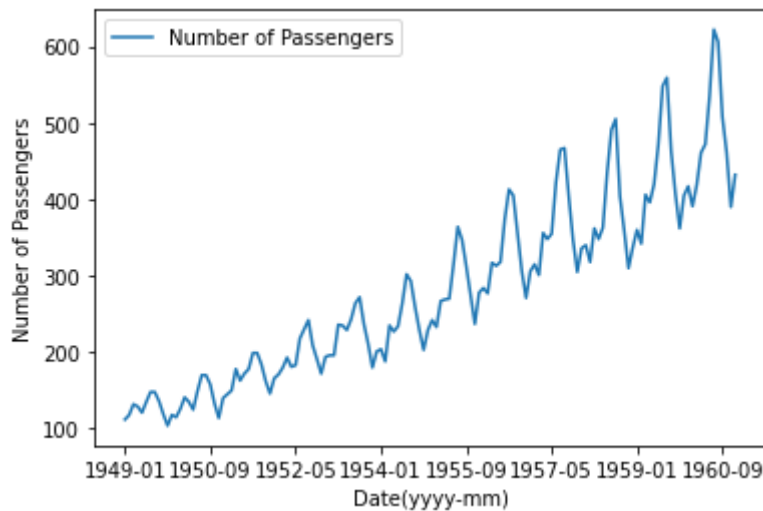

```
#Reading Time-series data into Pandas dataframe
import pandas as pd
PassengerData = pd.read_csv('/content/AirPassengers.csv')
PassengerData.columns=['Date (yyyy-mm)', 'Number of Passengers']
PassengerData.head(21)
```

	Date (yyyy-mm)	Number of Passengers
0	1949-01	112
1	1949-02	118
2	1949-03	132
3	1949-04	129
4	1949-05	121
5	1949-06	135
6	1949-07	148
7	1949-08	148
8	1949-09	136
9	1949-10	119
10	1949-11	104
11	1949-12	118
12	1950-01	115
13	1950-02	126
14	1950-03	141
15	1950-04	135
16	1950-05	125
17	1950-06	149
18	1950-07	170
19	1950-08	170
20	1950-09	158

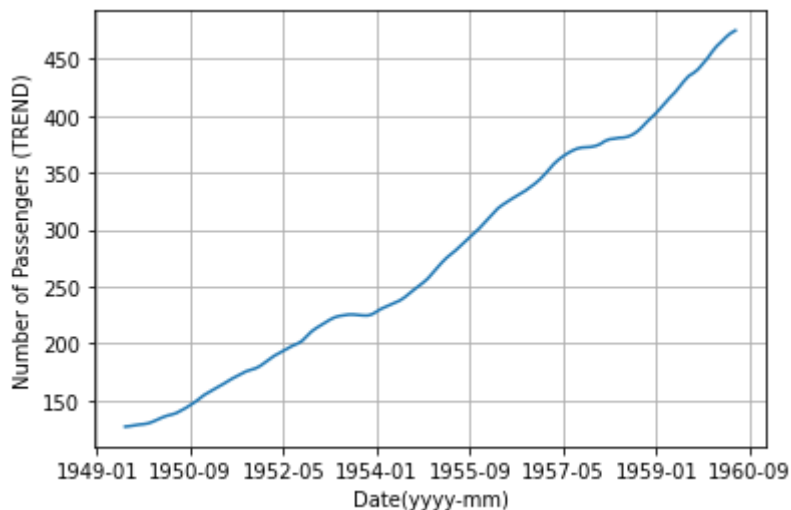
```
#PLOTING TIME-SERIES DATA using plot() function
import pandas as pd
import matplotlib.pyplot as plt
PassengerData = pd.read_csv('/content/AirPassengers.csv')
PassengerData.columns=['Date(yyyy-mm)', 'Number of Passengers']
PassengerData.plot(x='Date(yyyy-mm)', y='Number of Passengers')

title=('Number of US Airline passengers from 1949 to 1960')
```

```
plt.ylabel('Number of Passengers')
plt.show()
```



```
#ETS Decomposition-MULTIPLICATIVE MODEL
#Extracting TREND component
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose as sm
PassengerData = pd.read_csv('/content/AirPassengers(1).csv')
PassengerData.columns=['Date(yyyy-mm)', 'Number of Passengers']
PassengerData.set_index('Date(yyyy-mm)', inplace=True)
multiplicative_decomposition = sm(PassengerData['Number of Passengers'], model='multiplica
multiplicative_decomposition.trend.plot()
plt.ylabel('Number of Passengers (TREND)')
plt.grid()
```

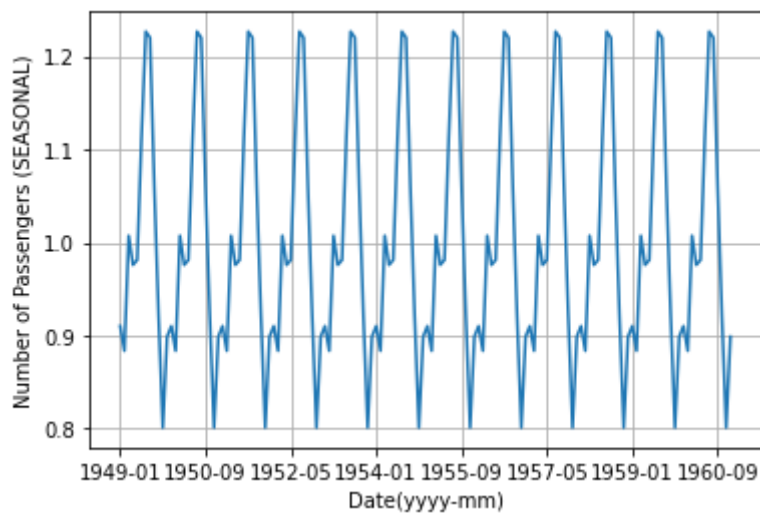


```
#Extracting SEASONAL component
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose as sm
PassengerData = pd.read_csv('/content/AirPassengers(1).csv')
PassengerData.columns=['Date(yyyy-mm)', 'Number of Passengers']
PassengerData.set_index('Date(yyyy-mm)', inplace=True)
multiplicative_decomposition = sm(PassengerData['Number of Passengers'], model='multiplica
```

```

multiplicative_decomposition.seasonal.plot()
plt.ylabel('Number of Passengers (SEASONAL)')
plt.grid()

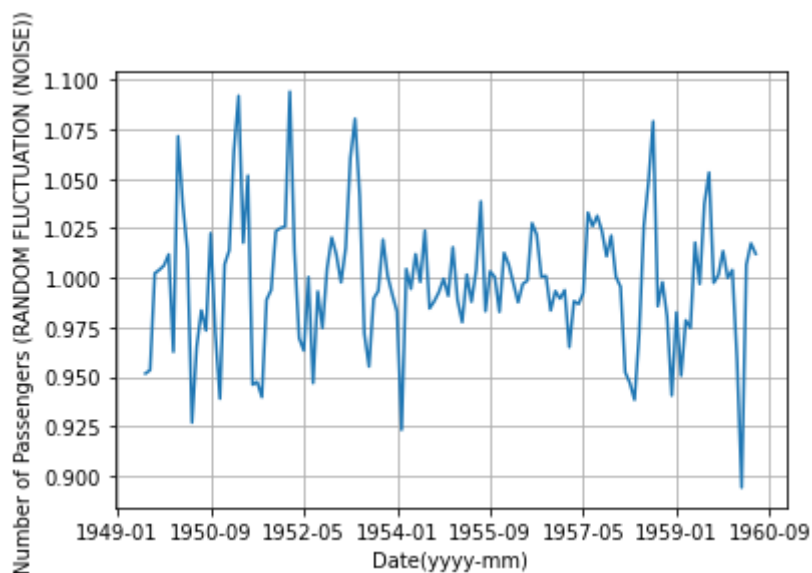
```



```

#Extracting RESIDUAL component
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose as sm
PassengerData = pd.read_csv('/content/AirPassengers(1).csv')
PassengerData.columns=['Date(yyyy-mm)', 'Number of Passengers']
PassengerData.set_index('Date(yyyy-mm)', inplace=True)
multiplicative_decomposition = sm(PassengerData['Number of Passengers'], model='multiplica
multiplicative_decomposition.resid.plot()
plt.ylabel('Number of Passengers (RANDOM FLUCTUATION (NOISE))')
plt.grid()

```



```

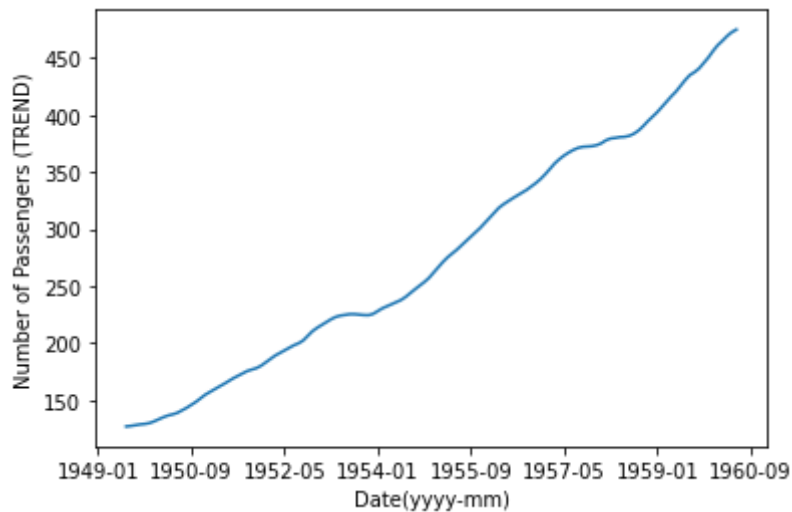
#ETS Decomposition-ADDITIVE MODEL
#Extracting TREND component
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose as sm
PassengerData = pd.read_csv('/content/AirPassengers(1).csv')

```

```

PassengerData.columns=['Date(yyyy-mm)', 'Number of Passengers']
PassengerData.set_index('Date(yyyy-mm)', inplace=True)
Additive_decomposition = sm(PassengerData['Number of Passengers'], model='additive', period=12)
Additive_decomposition.trend.plot()
plt.ylabel('Number of Passengers (TREND)')
plt.show()

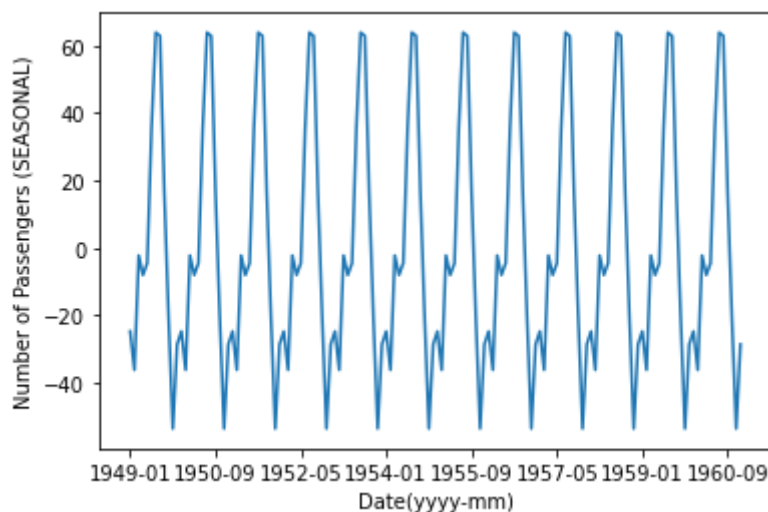
```



```

#Extracting SEASONAL component
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose as sm
PassengerData = pd.read_csv('/content/AirPassengers(1).csv')
PassengerData.columns=['Date(yyyy-mm)', 'Number of Passengers']
PassengerData.set_index('Date(yyyy-mm)', inplace=True)
Additive_decomposition = sm(PassengerData['Number of Passengers'], model='additive', period=12)
Additive_decomposition.seasonal.plot()
plt.ylabel('Number of Passengers (SEASONAL)')
plt.show()

```



```

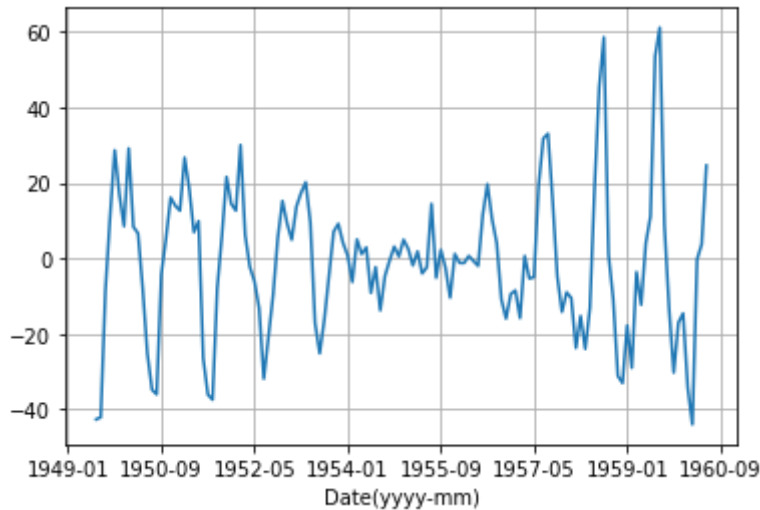
#Extracting RESIDUAL component
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose as sm

```

```

PassengerData = pd.read_csv('/content/AirPassengers(1).csv')
PassengerData.columns=['Date(yyyy-mm)', 'Number of Passengers']
PassengerData.set_index('Date(yyyy-mm)', inplace=True)
Additive_decomposition = sm(PassengerData['Number of Passengers'], model='additive', perio
Additive_decomposition.resid.plot()
plt.grid()
plt.show()

```



```

#SuperStore-SalesDataSet
#Reading Time-series data into Pandas dataframe
import pandas as pd
PassengerData = pd.read_csv('/content/SuperStore-SalesDataSet.csv')
PassengerData.columns=['Row ID', 'Order ID', 'Order Date(yyyy-mm)', 'Ship Date(yyyy-mm)',
'City', 'State', 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category']
PassengerData.head()
pd.options.display.max_rows = 999

```

```
df = pd.read_csv('/content/SuperStore-SalesDataSet.csv')
```

```
print(df)
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	
0	1	CA-2017-152156	08/11/2017	11/11/2017	Second Class	
1	2	CA-2017-152156	08/11/2017	11/11/2017	Second Class	
2	3	CA-2017-138688	12/06/2017	16/06/2017	Second Class	
3	4	US-2016-108966	11/10/2016	18/10/2016	Standard Class	
4	5	US-2016-108966	11/10/2016	18/10/2016	Standard Class	
...	...	...	...	...	...	
9795	9796	CA-2017-125920	21/05/2017	28/05/2017	Standard Class	
9796	9797	CA-2016-128608	12/01/2016	17/01/2016	Standard Class	
9797	9798	CA-2016-128608	12/01/2016	17/01/2016	Standard Class	
9798	9799	CA-2016-128608	12/01/2016	17/01/2016	Standard Class	
9799	9800	CA-2016-128608	12/01/2016	17/01/2016	Standard Class	

	Customer ID	Customer Name	Segment	Country	City	
0	CG-12520	Claire Gute	Consumer	United States	Henderson	
1	CG-12520	Claire Gute	Consumer	United States	Henderson	
2	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	
3	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	
4	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	
...	...	...	...	...	...	
9795	SH-19975	Sally Hughsby	Corporate	United States	Chicago	

9796	CS-12490	Cindy Schnelling	Corporate	United States	Toledo
9797	CS-12490	Cindy Schnelling	Corporate	United States	Toledo
9798	CS-12490	Cindy Schnelling	Corporate	United States	Toledo
9799	CS-12490	Cindy Schnelling	Corporate	United States	Toledo

	State	Postal Code	Region	Product ID	Category \
0	Kentucky	42420.0	South	FUR-BO-10001798	Furniture
1	Kentucky	42420.0	South	FUR-CH-10000454	Furniture
2	California	90036.0	West	OFF-LA-10000240	Office Supplies
3	Florida	33311.0	South	FUR-TA-10000577	Furniture
4	Florida	33311.0	South	OFF-ST-10000760	Office Supplies
...	...	...	...	...	...
9795	Illinois	60610.0	Central	OFF-BI-10003429	Office Supplies
9796	Ohio	43615.0	East	OFF-AR-10001374	Office Supplies
9797	Ohio	43615.0	East	TEC-PH-10004977	Technology
9798	Ohio	43615.0	East	TEC-PH-10000912	Technology
9799	Ohio	43615.0	East	TEC-AC-10000487	Technology

	Sub-Category	Product Name	Sales
0	Bookcases	Bush Somerset Collection Bookcase	261.9600
1	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400
2	Labels	Self-Adhesive Address Labels for Typewriters b...	14.6200
3	Tables	Bretford CR4500 Series Slim Rectangular Table	957.5775
4	Storage	Eldon Fold 'N Roll Cart System	22.3680
...	...	...	...
9795	Binders	Cardinal HOLDit! Binder Insert Strips,Extra St...	3.7980
9796	Art	BIC Brite Liner Highlighters, Chisel Tip	10.3680
9797	Phones	GE 30524EE4	235.1880
9798	Phones	Anker 24W Portable Micro USB Car Charger	26.3760
9799	Accessories	SanDisk Cruzer 4 GB USB Flash Drive	10.3840

[9800 rows x 18 columns]

#PLOTING TIME-SERIES DATA using plot() function

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
StoreData = pd.read_csv('/content/SuperStore-SalesDataSet.csv')
```

```
PassengerData.columns=['Row ID', 'Order ID', 'Order Date(yyyy-mm)', 'Ship Date(yyyy-mm)', 'City', 'State', 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category']
PassengerData.plot(x='State', y='Sales')
```

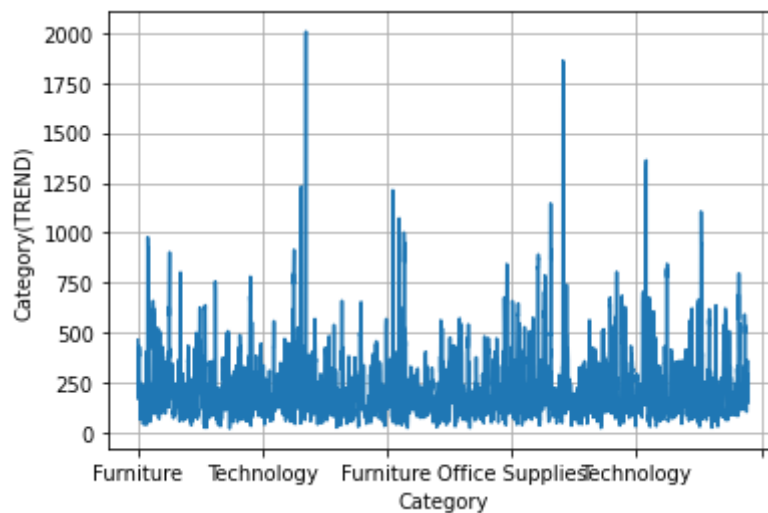
```
title=('SuperStore-Sales Data')
```

```
plt.ylabel('Sales')
```

```
plt.show()
```

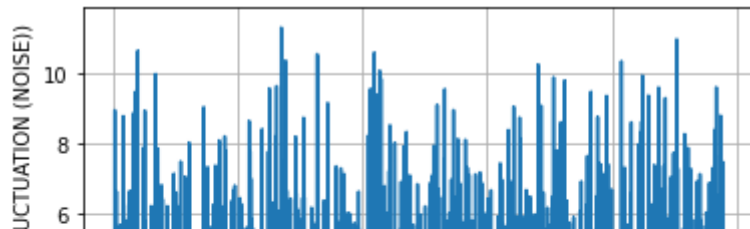


```
#ETS Decomposition-MULTIPLICATIVE MODEL
#Extracting TREND component
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose as sm
StoreData = pd.read_csv('/content/SuperStore-SalesDataSet.csv')
StoreData.columns=['Row ID', 'Order ID', 'Order Date(yyyy-mm)', 'Ship Date(yyyy-mm)',
'City', 'State', 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category']
StoreData.set_index('Category',inplace=True)
multiplicative_decomposition = sm(StoreData['Sales'], model='multiplicative', period=12)
multiplicative_decomposition.trend.plot()
plt.ylabel('Category(TREND)')
plt.grid()
```

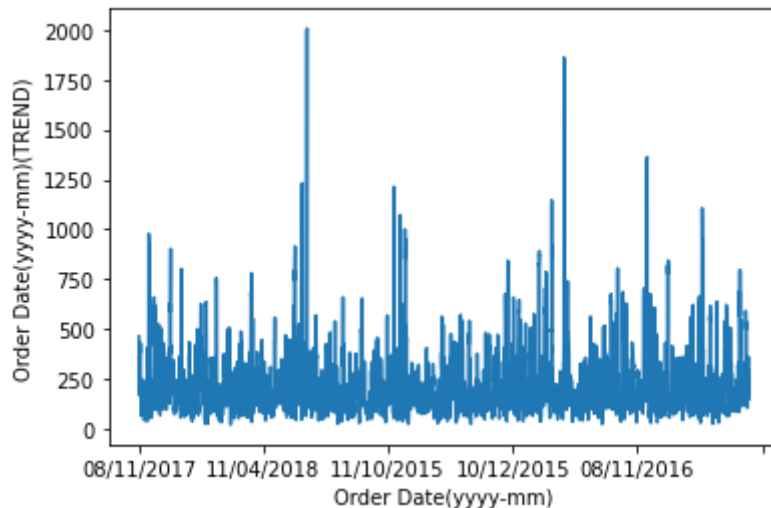


```
#Extracting RESIDUAL component
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose as sm
StoreData = pd.read_csv('/content/SuperStore-SalesDataSet (1).csv')
StoreData.columns=['Row ID', 'Order ID', 'Order Date(yyyy-mm)', 'Ship Date(yyyy-mm)', 'S
'City', 'State', 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category', 'Pro
StoreData.set_index('Region',inplace=True)
multiplicative_decomposition = sm(StoreData['Sales'], model='multiplicative', period=12)
multiplicative_decomposition.resid.plot()
plt.ylabel('Region((RANDOM FLUCTUATION (NOISE)))')
plt.grid()
```

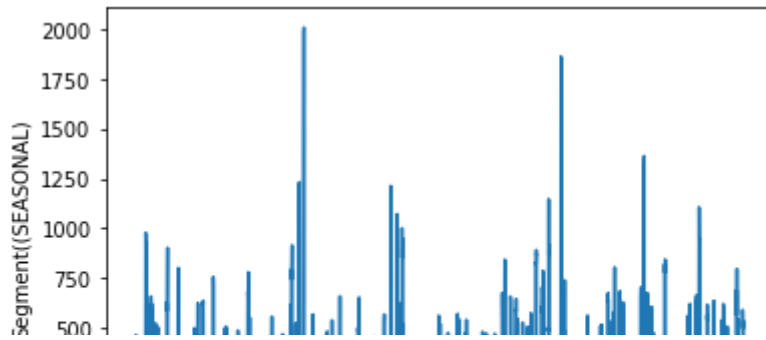




```
#ETS Decomposition-ADDITIVE MODEL
#Extracting TREND component
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose as sm
StoreData = pd.read_csv('/content/SuperStore-SalesDataSet (1).csv')
StoreData.columns=['Row ID', 'Order ID', 'Order Date(yyyy-mm)', 'Ship Date(yyyy-mm)', 'S
'City', 'State', 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category', 'Pro
StoreData.set_index('Order Date(yyyy-mm)',inplace=True)
Additive_decomposition = sm(StoreData['Sales'], model='additive', period=12)
Additive_decomposition.trend.plot()
plt.ylabel('Order Date(yyyy-mm)(TREND)')
plt.show()
```



```
#Extracting SEASONAL component
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose as sm
StoreData = pd.read_csv('/content/SuperStore-SalesDataSet (1).csv')
StoreData.columns=['Row ID', 'Order ID', 'Order Date(yyyy-mm)', 'Ship Date(yyyy-mm)',
'City', 'State', 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category'
StoreData.set_index('Segment',inplace=True)
Additive_decomposition = sm(StoreData['Sales'], model='additive', period=12)
Additive_decomposition.trend.plot()
plt.ylabel('Segment((SEASONAL))')
plt.show()
```

#Extracting RESIDUAL component

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from statsmodels.tsa.seasonal import seasonal_decompose as sm
```

```
StoreData = pd.read_csv('/content/SuperStore-SalesDataSet (1).csv')
```

```
StoreData.columns=['Row ID', 'Order ID', 'Order Date(yyyy-mm)', 'Ship Date(yyyy-mm)',  
'City', 'State', 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category']
```

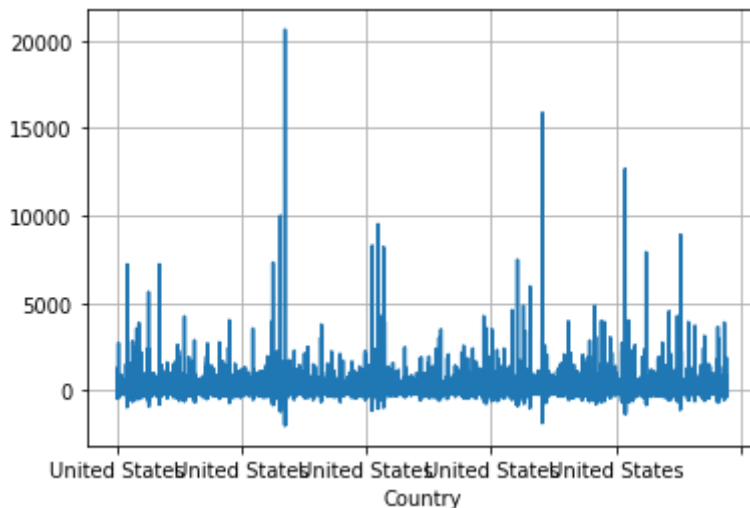
```
StoreData.set_index('Country',inplace=True)
```

```
Additive_decomposition = sm(StoreData['Sales'], model='additive', period=12)
```

```
Additive_decomposition.resid.plot()
```

```
plt.grid()
```

```
plt.show()
```



[Colab paid products](#) - [Cancel contracts here](#)

